07.02.2019 (1)

Derleyici Tasarımı

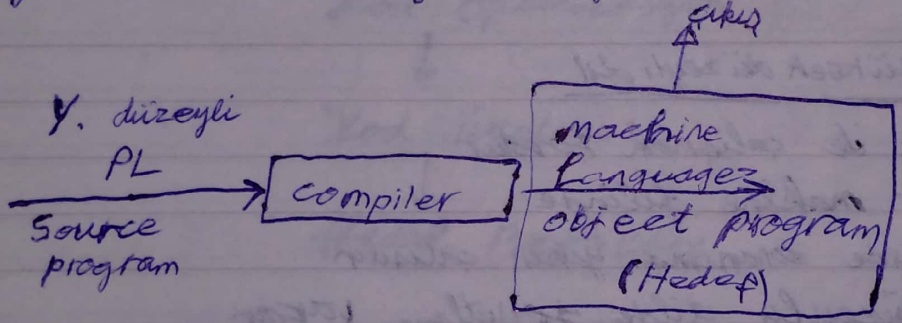
$$\left((x - x_0)^2 + (x - x_1)^2 \right)^{\frac{1}{2}}$$

Hangisi ilk işlenir?

if (A[K] < max) sum = 0 else A[K] = 0;

makineye yaklaştıkça hız artar, ama okunabilirlik azalır. Okunabilirlik ise feda edilemez.

İlk derleyici Fortran dili için IBM'de çalışan John B tarafından yazılmıştır.



C++

A = B + C * D

giriş

Assembly

```
LOD  R1, C
MUL  R1, D
STO  R1, T1
LOD  R1, B
ADD  R1, T1
STO  R1, T2
MOV  A , T2
```

İşlem önceliğinin belirlenmesi için grameğin iyi tanımlanması gerekiyor

ÖRNEKLER:

while (x < a+b) x = 2 * x

```
L1  LOD  R1, A
    ADD  R1, B
    STO  R1, T1
    CMP  x, T1
    BL   L2
    B    L3
L2  LOD  R1, "=2"
    MUL  R1, x
    STO  R1, x
    BL1
L3  SON
```

→ Çıkış

giriş

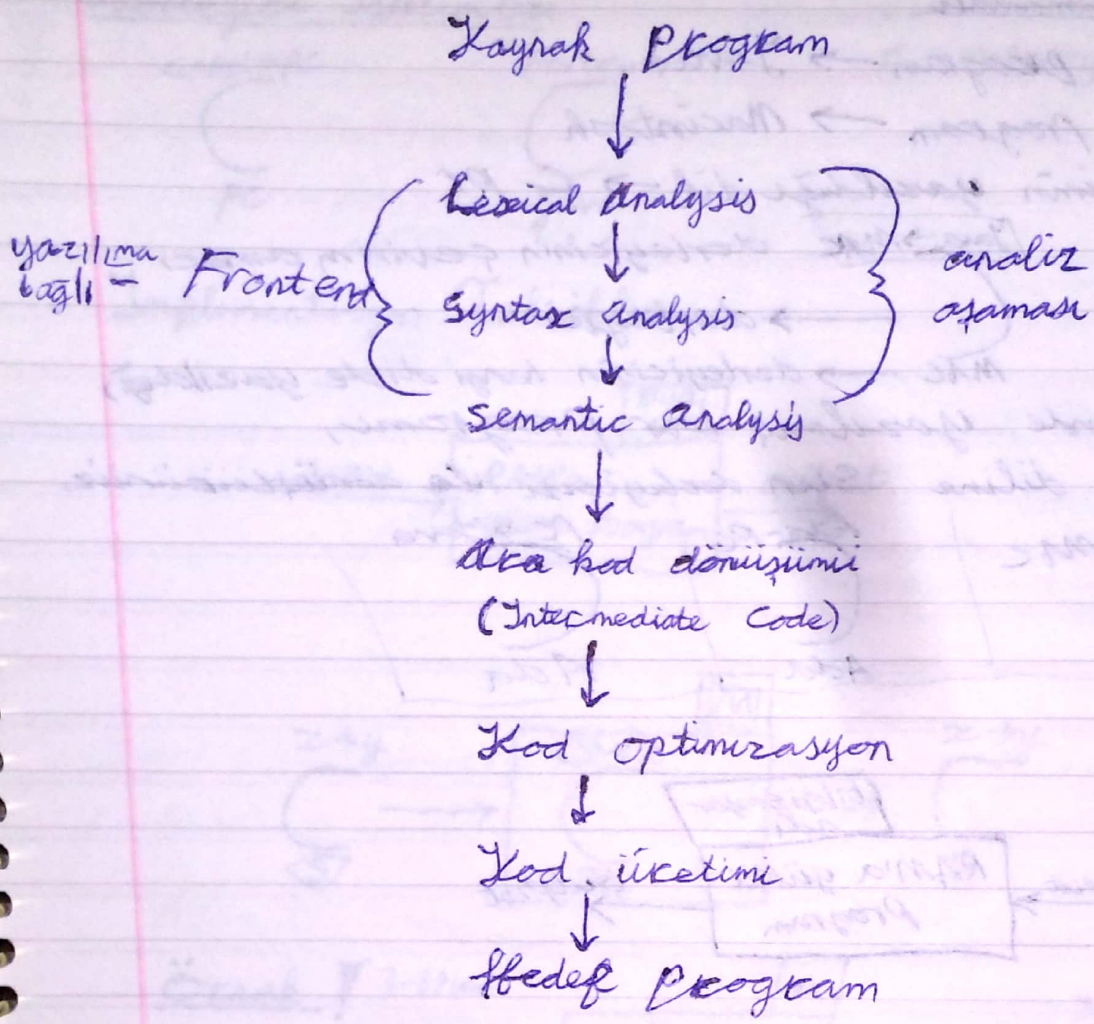
Derleyici — yüksek düzeyli dil

1) makine dili ile çalışmak zordur
2) Bakım zor makine dilinde
3) Makine dilinde donanıma yakın çalışılır
4) yüksek düzeyli dilde soyutlama vardır
5) En verimli dil makine dilidir

kaynak → interpreter → Çıkış

Derleyici tüm programı işler, makine kodunu çevirir, ardından girişleri alır.
Yorumlayıcı bir satır işler, o satırı işlemeden devam etmez.

!! vizede



ÖRNEK: for(i=1; i<=4; i++)
cout << i*3;

a) Programın yorumlayıcı çıktısı?
b) Programın derleyici çıktısı?

giriş

Derleyici

```
     LOD R1, "=4"
     STO R1, T1
L1   CMP i, T1
     BH  L2
     LOD R1, i
     MUL R1, "3"
     STO R1, T2
     PUSH T2
     CALL WRITE
     ADD  i="1"
     B    L1
L2
```

→ çıkış

Yorumlayıcı

3, 6, 9, 12

Derleyici Elemanları

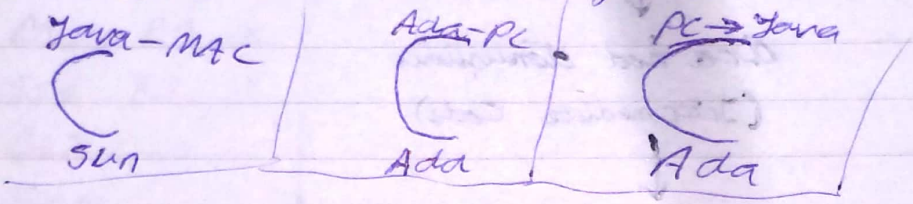
1) Soucce program → Java
2) Object program → Macintosh
3) Derleyicinin yazıldığı dil → C pr.

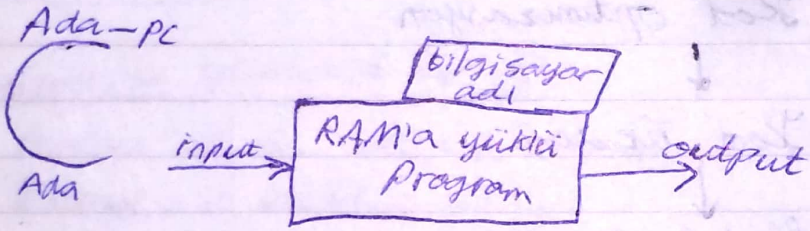
Java→MAC derleyicinin çevirim amacı

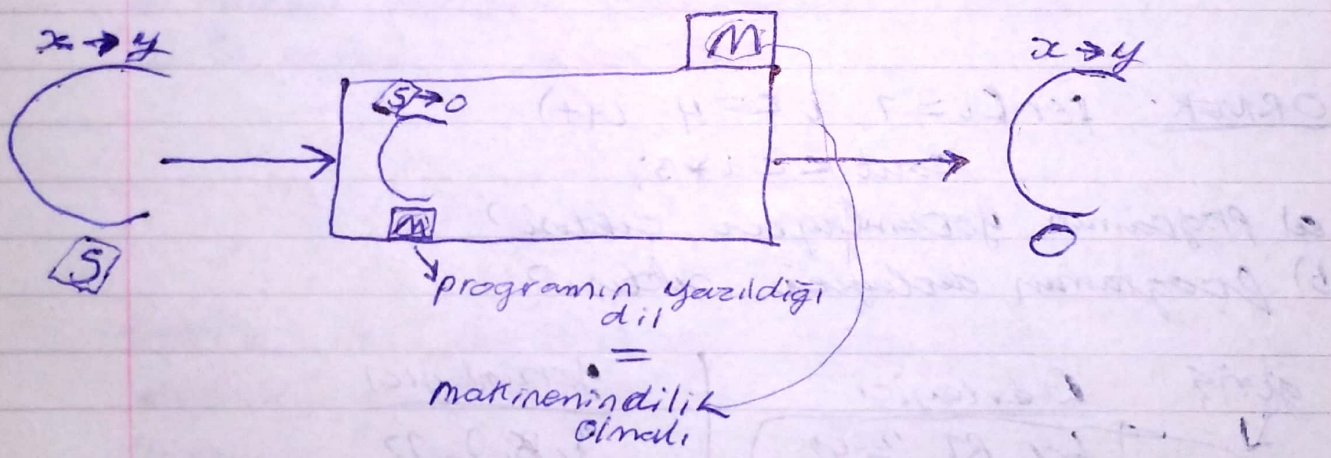
→ derleyici

MAC → derleyicinin hangi dilde yazıldığı

ÖRNEK: Java dilinde yazılmış bir programı, Macintosh diline Sun derleyicisi ile dönüştürünüz.



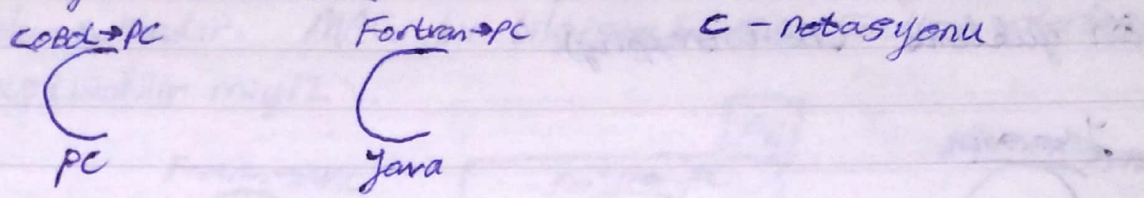




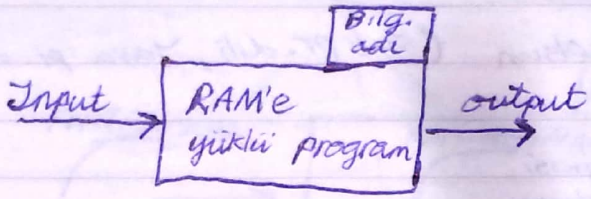
x dilinde yazılmış programı, y diline çeviren ve S makinesinde çalışan derleyiciyi, O makinesinde çalışabilecek hale getiriliyor.

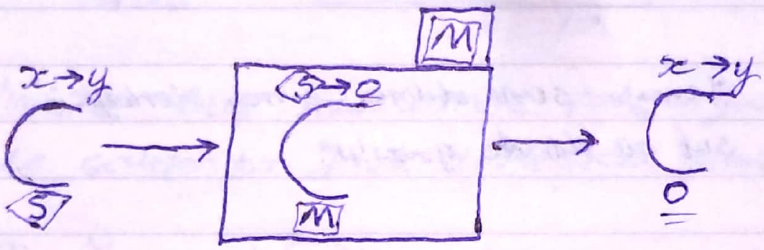
14.02.2019 (2)

Derleyici İşletimleri



Implementasyon Tekrikleri



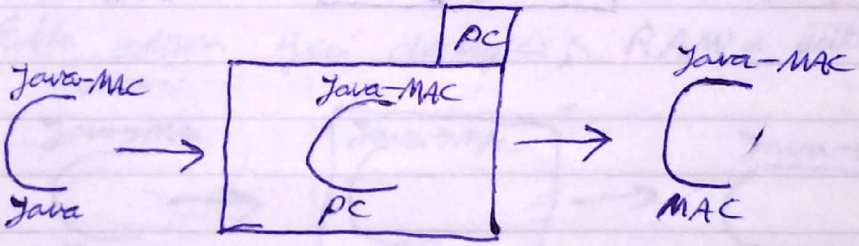


Örnek ! 7-8 Puan

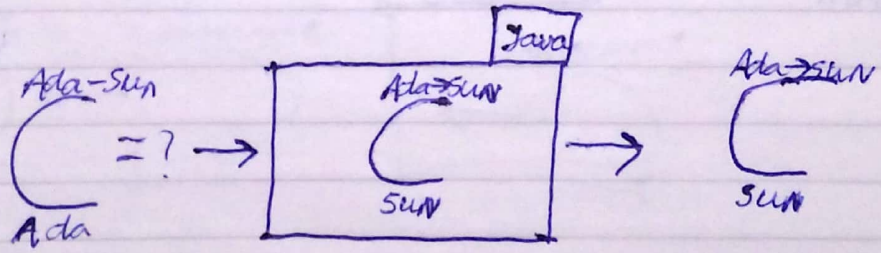
a)



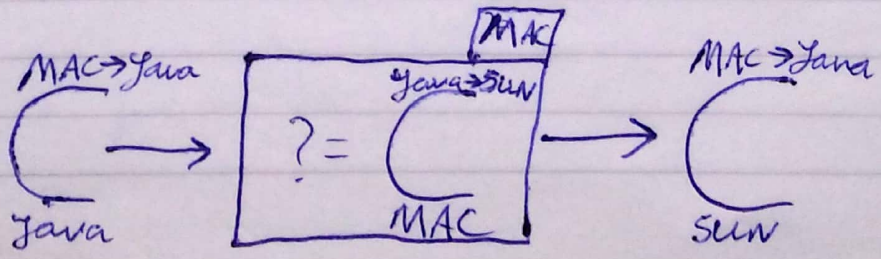
b)



c)

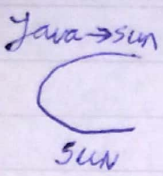


d)

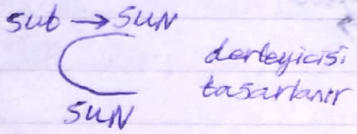


Implementasyon Teknikleri

ön yükleme (Bootstrapping)

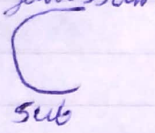


Adım 1: Sub C Java olsun (Sub pr. dili, Java pr. dilinin alt kümesi)

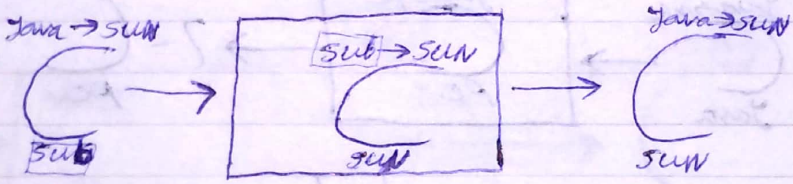


Adım 2: Java→SUN

Javayı SUN diline çeviren derleyici, sub alt dilinde yazılır.

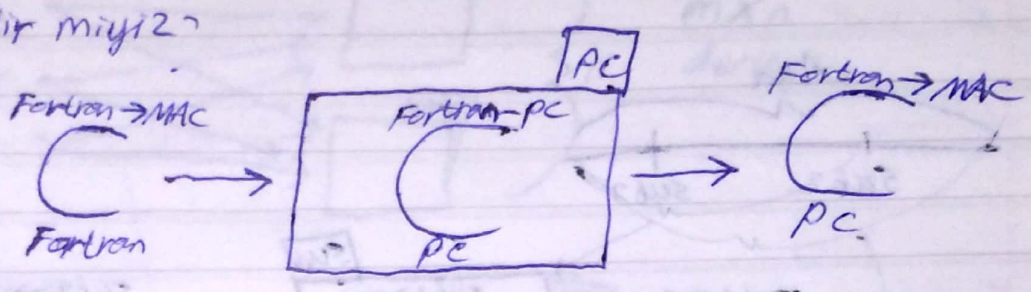


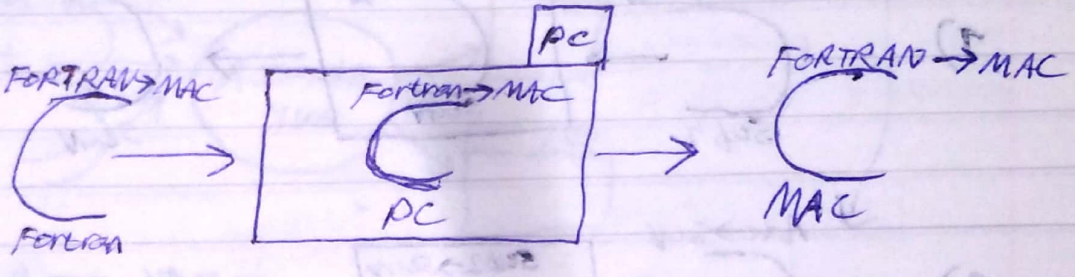
Adım 3: adım 1'de tasarlan derleyici RAM'e yüklenir



ÖRNEK- Elimizde bir Pentium bilgisayar ve Fortran derleyicisi bulunmaktadır. MAC bir bilgisayar için Fortran derleyicisi oluşturabilir miyiz?

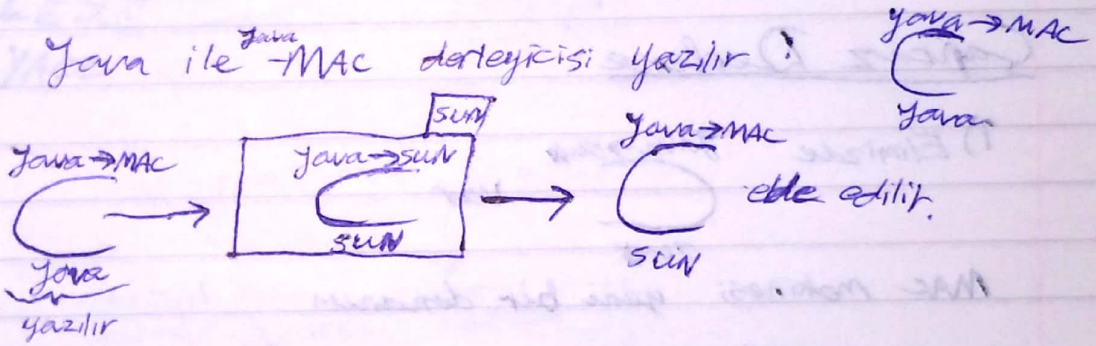
7-8 puan



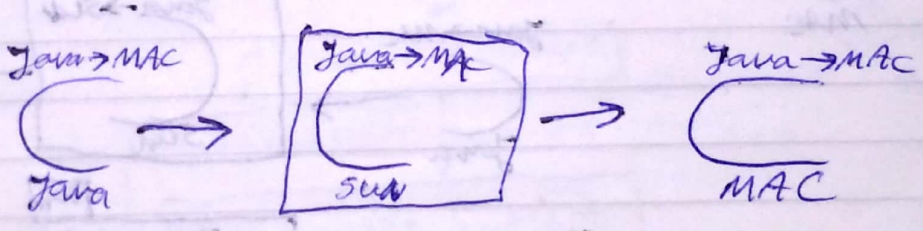


ÖRNEK: Elimizde Sun makinesinde çalışan Java derleyicisi var. MAC'de çalışan bir Java derleyicisi oluşturmak mümkün müdür?

Adım 1: Java ile Java→MAC derleyicisi yazılır!

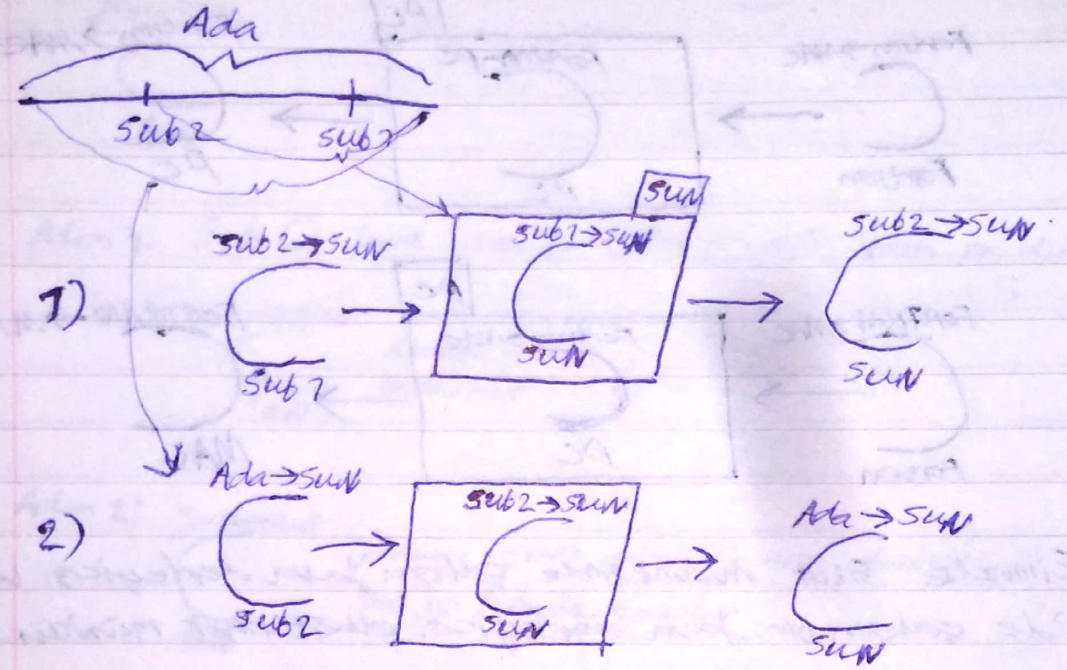


Adım 2: Elde edilen yeni derleyici, RAM'e yüklenir



ÖRNEK: Tekrarlı iki ön yüklemle gerçekleştirilebilir mi?

Sub1 C Sub2 C Ada



Çapraz Derleme

1) Elimizde Java→SUN var

SUN

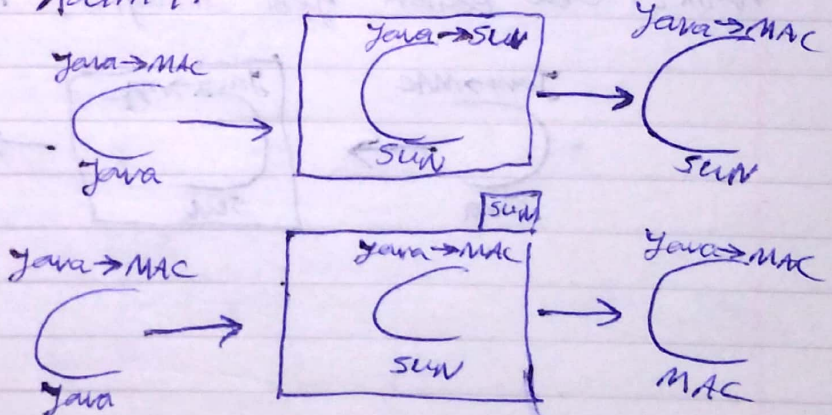
MAC makinesi yeni bir donanım

Java→MAC

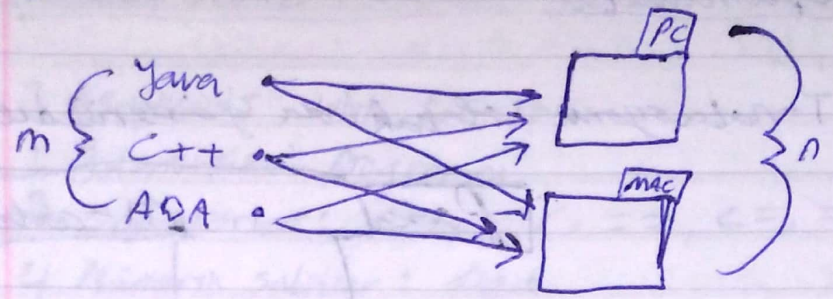
MAC

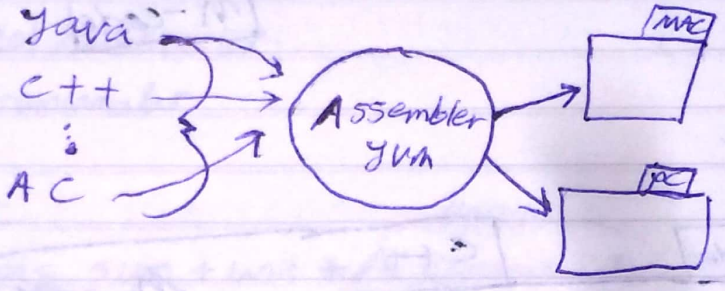
?

Adım 1:



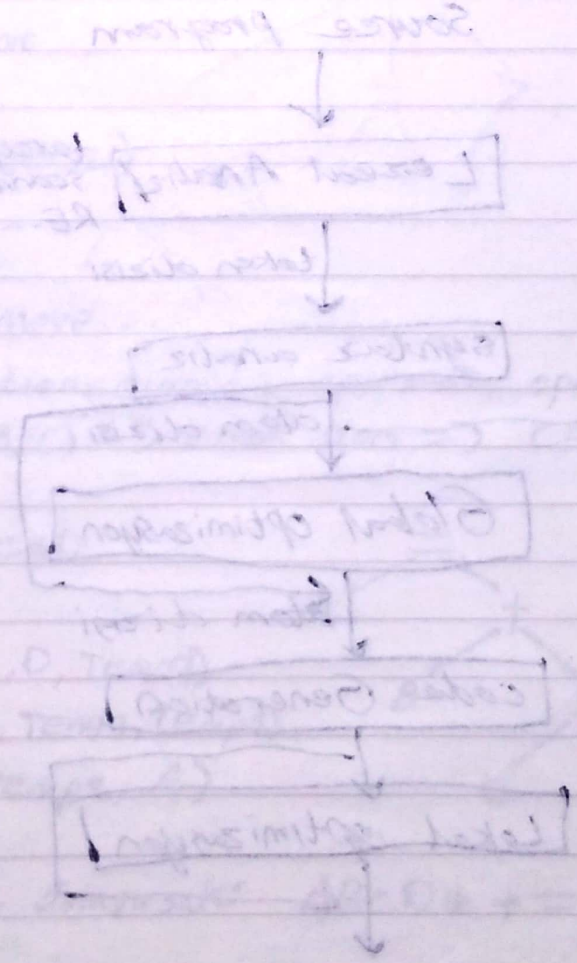
③ ARA KOD





Apple II
PDP-8
} P-code

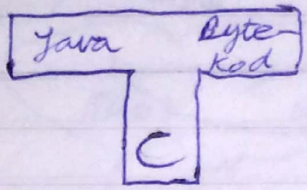
4) i) LEX
ii) YACC



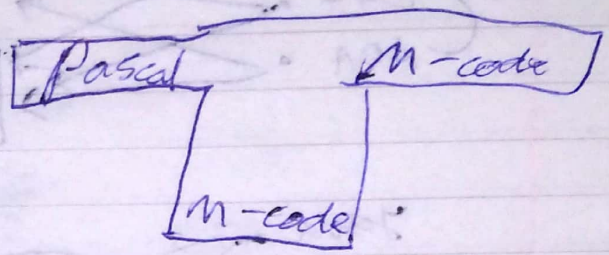
21.02.2019 (3)

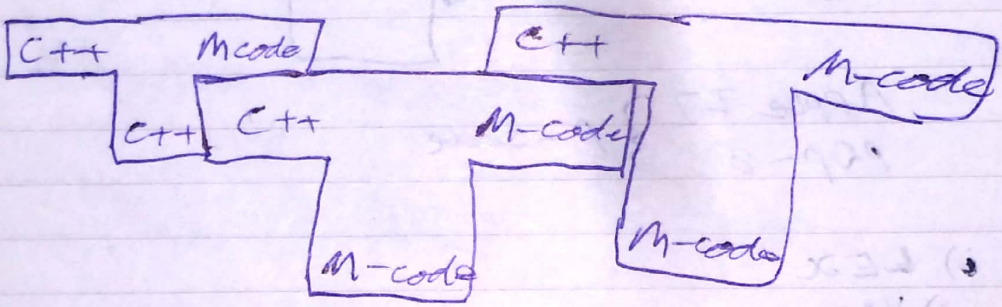
Derleyicinin Aşamaları

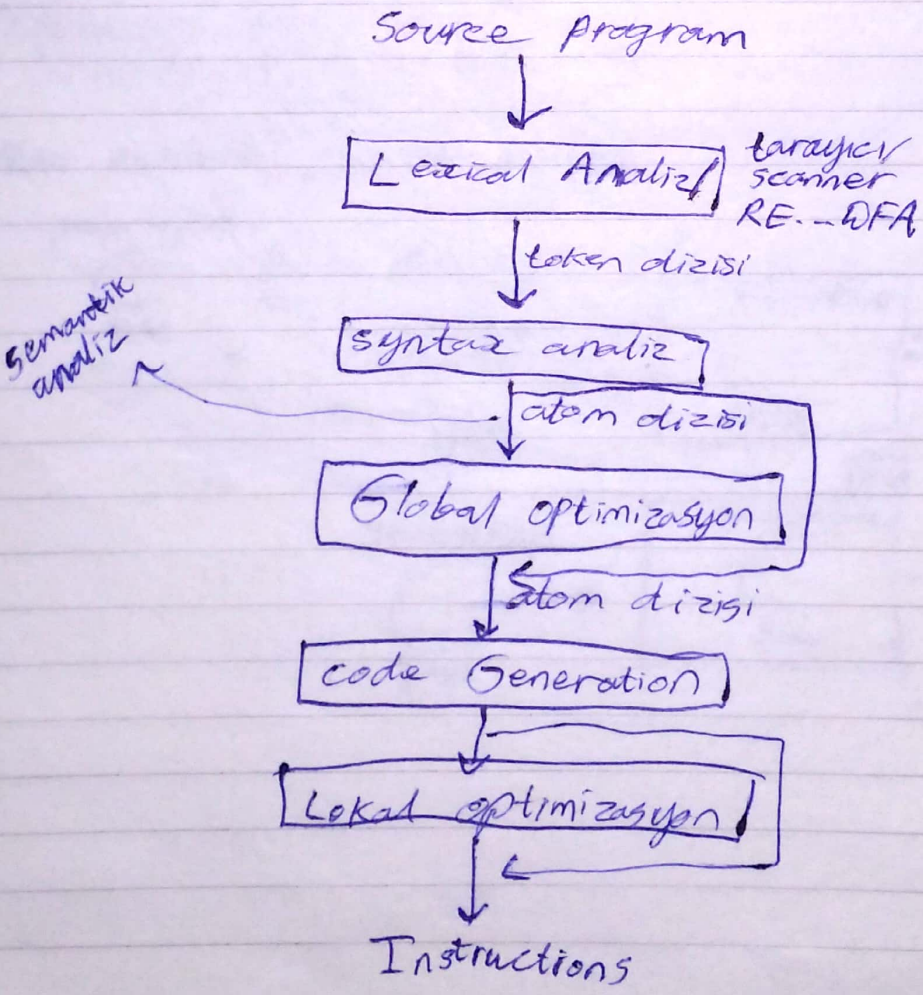
C-notasyonu, T-notasyonu olarak da gösterilebilir:



C dilinde yazılmış, Java'dan bytekod'a dönüştüren derleyici







Lexical Analiz / Scanner

1. Keywords : while, for, if...
2. Identifier: programcı
3. Operatörler: +, −, *, /, ==, <=, =>, ...
4. Nümerik sabitler : digit
5. Karakter Sabitler
6. Özel karakterler
7. Yorumlamalar

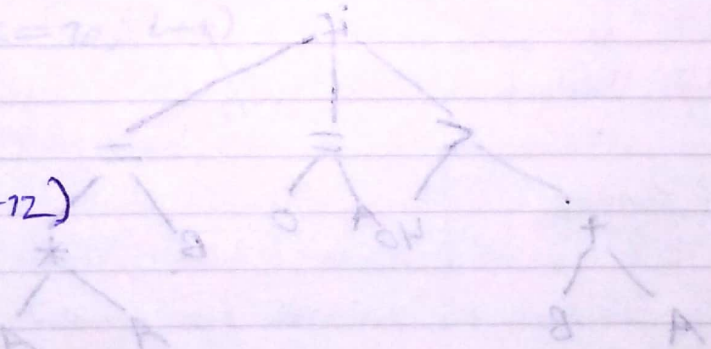
Örnek:

Sum = sum + unit */* toplamlar */ 1.5e-12;

(yorum)

a) Lexical analiz çıkışı nedir?

Identifier (sum)
assign (=)
identifier (sum)
operator +
identifier unit
operator *
Numerical Unit (1.5e-12)



b) Syntax analiz

MULT, ADD, MOVE...

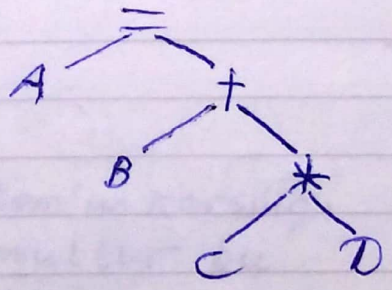
ATOM: (operation, operand1, operand2, operand3)

(ADD, A, B, C) => örnek atom => C = a+b

ÖRNEK:

A = B + C * D

(MULT, C, D, TEMP1)
(ADD, B, TEMP1, TEMP2)
(MOVE, TEMP2, A)



Ağacı postfix dolaşırsak: ABCD * + =

1,2,3,4,5,6

(TEST, A, <=, B, L2) A <= B ise, Level 2'ye git

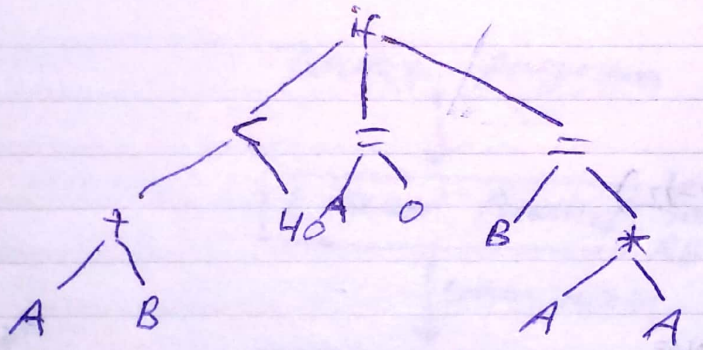
if (A<=B) (T)

ÖRNEK:

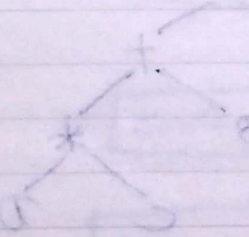
while (A<=B) A=A+1;

(LBL, L1)
(TEST, A, <=, B, L2)
(JMP, L3)
(LBL, L2)
(ADD, A, 1, A)
(JMP, L1)
(LBL, L3)

ÖRNEK: if (A+B) < 40 A=0; else B= A*A;



Semantik analiz aşaması için parse ağaçları da kullanılabilir.



Global Optimizasyon

Makineden bağımsız optimizasyon (donanım bağımsız)
Ara dizinde optimizasyon yapılması

Stmt 1	for(i=1; i<=100.000; i++)
go to L1	{
Stmt 2	x = sqrt(y)
Stmt 3	cout << x+i << endl;
label 2 Stmt 4	}

ÖRNEK:

```
for(i=1; i<=10; i++)
{
    x = i;
    y = x/2;
    a[i]=x;
}
```

yerine

```
for(i=1; i<=10; i++)
    a[i]= i;
x=10;
y = x/2;
```

Code Generation

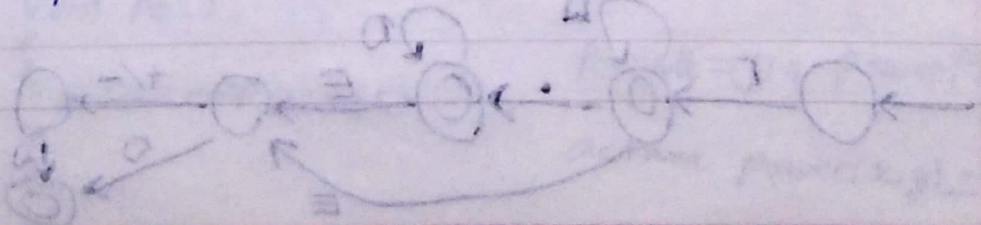
Kod üretimi kısmı makineye bağlıdır.

ADD:
1) Birinci operand
2) İkinci operand
3) Sonucun saklanması

(ADD, A, B, TEMP)

```
LOD R1, A
ADD R1, B
STO R1, TMP
```

} 1 atom'un karşılığı 3 komuttur bu örnekte



ÖRNEK:

(ADD A, B, Temp1)
(TEST, A, --, B, L1)
(MOVE, Temp1, A)
(LBL, L1)
(MOVE, Temp1, B)

```
LOD R1, A
ADD R1, B
STO R1, Temp1
CMP A, B
BE L1
MOV A, Temp1
L1 MOV B, Temp1
```

Local Optimasation

Makine/donanım bağımlı optimizasyon.
Üretilen kodlarda optimizasyon

```
LOD R1, A
ADD R1, B
* STO R1, Temp1
* LOD Temp1, R1
ADD R1, C
STO R1, Temp2
```

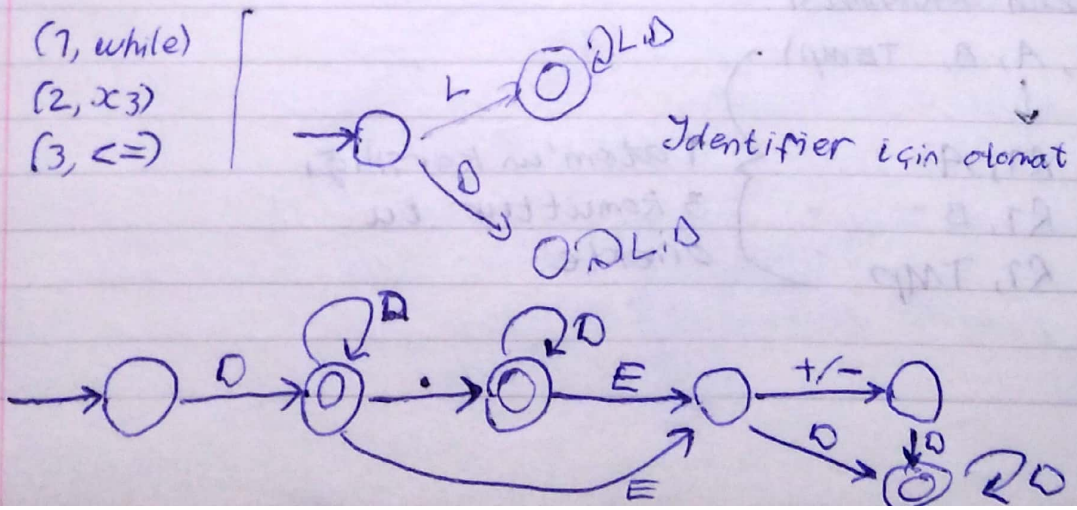
} gereksiz kod, çıkarılabilir

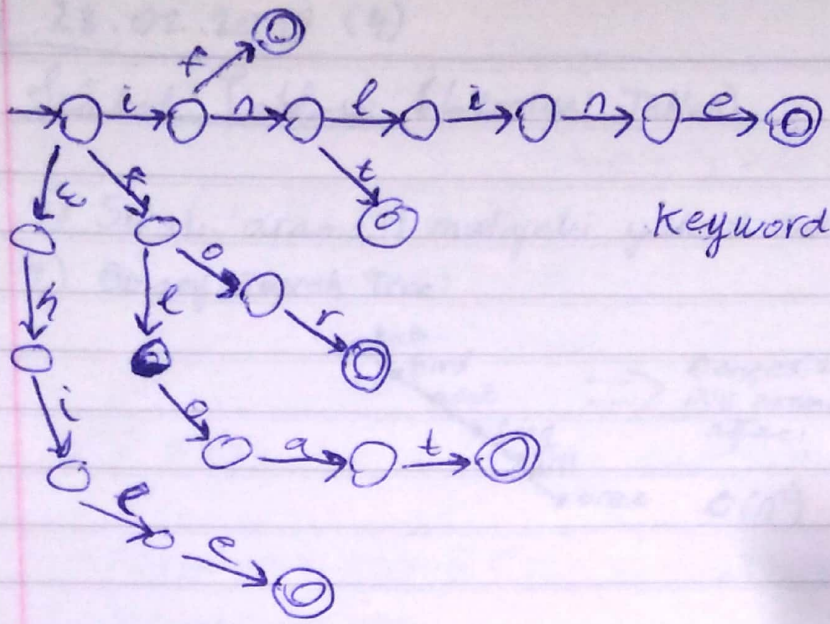
Lexical Analiz

Regüler ifade bir dil tanımlar,
DFA ise çok hızlı bir şekilde bir ifadenin bir dile ait olup olmadığını belirler.

While (x3 <= 2.2E+3 - t2) hesap (x3); // !
1 6 2 3 2 6 6 6

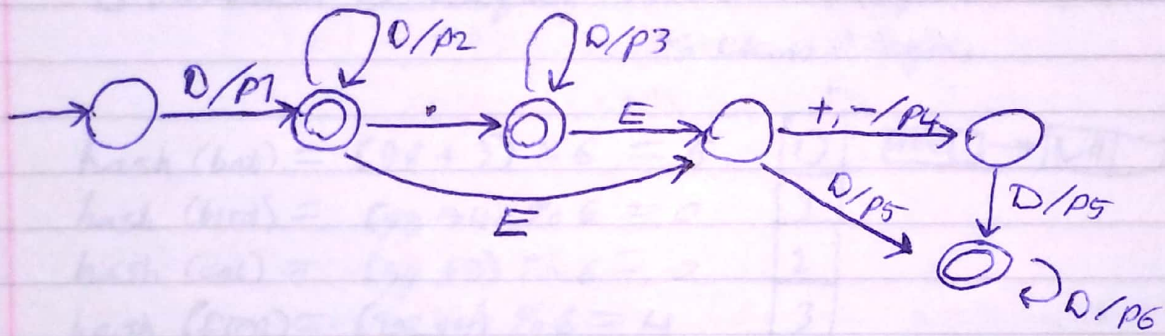
(1, while)
(2, x3)
(3, <=)





Keyword'ler için otomat

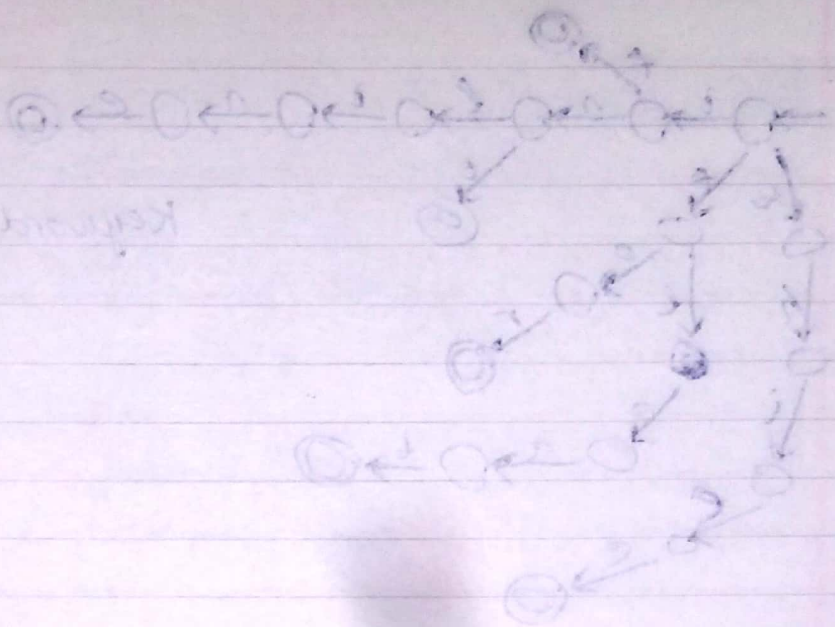
Nümerik katarları okumak ve onları uygun bir veri yapısı formatına dönüştürmek için bir otomat tasarlayınız.

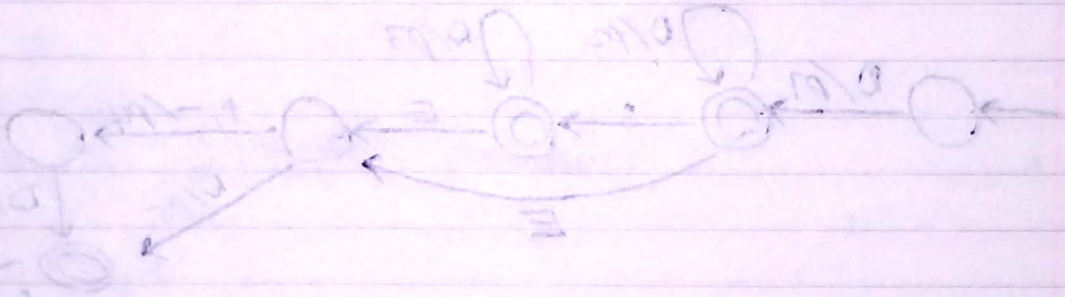



```
int P,N,D, Exp, Sign;
void p1()
{
  P=0
  N=D
  Exp=0
  Sign=+1
}
void p2()
{
  N = N*10+D;
}
Void p3()
{
  N=N*10 +D;
  P=p+1;
}
void p4()
{
  if(input=='-') then sign=-1;
}
Void p5()
{
  exp = D;
}
Void p6()
{
  exp=exp*10+D;
}
Result = N * power(10, sign*Exp)
açıklama power(x,y) = x^y
```

Lexical Tables

1) Sıralı arama
2) İkili arama
3) Hash tablosu



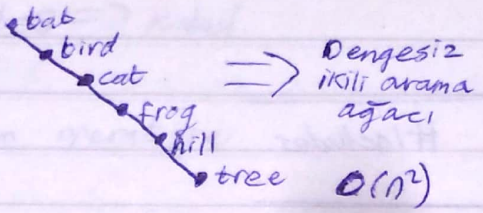


28.02.2019 (4)

Sembol Tablosu (Lexical Tables)

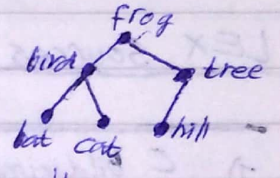
1) Sıralı arama : maliyeti yüksek $\Rightarrow O(n)$

2) Binary Search Tree:



Dengesiz ikili arama ağacı

$O(n^2)$



Dengeli ağaç :

$O(n \log n)$

3) Hash table:

{bat, bird, cat, frog, hill, tree}

Hash fonksiyonu: Sabit zaman

[(Kelimenin ilk harfinin ASCII karşılığı) + değişken uzunluğu] % eleman sayısı

hash (bat) = (98 + 3) % 6 = 5

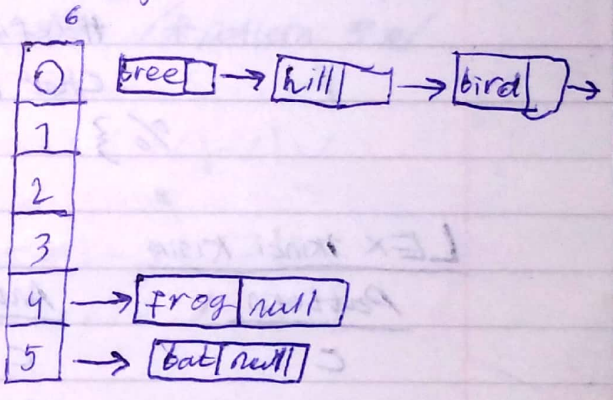
hash (bird) = (98 + 4) % 6 = 0

hash (cat) = (99 + 3) % 6 = 0

hash (frog) = (102 + 4) % 6 = 4

hash (hill) = (104 + 4) % 6 = 0

hash (tree) = (116 + 4) % 6 = 0



hash fonksiyonu seçilirken işlediği değerler ile mümkün oldukça unique değer üretebilmesi üzeri seçilmelidir.

Çakışma olduğu durumlarda bağlı listeler veya ikili ağaçlar kullanılabilir.

LEX:

→ yylex() → C dilinde

LEX → YACC

LEX Sources

1) C bildirimleri #includes
%% LEX fonksiyonları
2) LEX örüntüleri ve aksiyonlar => zorunlu kısım
3) C fonksiyonları

:

MAC ab*c
(makro)

foo {MAC} → foo {ab*c}

%{ #include "header.h"
#define MAX 100
char c
%}

LEX ikinci kısım

Pattern	Anlamı
c	c karakteri
\c	bir karakteri yinelemek
[cd]	c yada d
[a-z]	a ile z arasındaki küçük harfler
[^c]	c haricinde herhangi bir karakter
.	Enter haricinde herhangi bir karakter
^X	Satır başında X paterni
X$	Satır sonunda X paterni
X?	İsteğe bağlı olarak bir defa X
X*	Sıfır sayıda yada dilediğim kadar X
X+	1 sayıda yada dilediğim kadar X
Xy	X'den sonra kesinlikle y gelir
X/y	Sağ tarafında y varsa X
<5>X	5 başlangıç koşulum var ise X
X{m}	m adet X
X{m,n}	m ile n arası X

{name} makro

ÖRNEK:

[a-Z] */[0-9]

giriş: abc => ret

abc5 => kabul

/: Sağbağlam operatörü

Stdin → klavye

Stdout → ekran

aksiyon

%% [a-Z] + printf("alpha \n") /*pattern 1*/

[0-9] + printf("numeric\n") /*pattern 2*/

[a-Z0-9] + printf("alphanumeric \n") /*pattern 3*/

[\t] + printf("white space\n") /*pattern 4*/

. printf("special char\n") /*pattern 5*/

%%

abc → pattern 1 ile eşleşti

abc192 → pattern 3 ile eşleşti

LEX programının üçüncü kısmı

üçüncu kısım, sadece ikinci kısmındaki aksiyonlarda çağrılan C fonksiyonlarından oluşur.

Eğer yacc ile çalışmak isteniliyorsa, main() fonksiyonu eklenmelidir.

ÖRNEK:

1. kısım

INT [0-9]+

EXP([eE][+-]? {INT})

%% { int i;
float f;
%% }

2. kısım

%%

{INT} { sscanf(yytext, "%d", &i);
cout << "Int" << endl; }

örüntüye uyuyorsa → bunu gerçekleştir

{INT}\. {INT}? {EXP}?

{sscanf(yytext, "%f", &f);
; cout << "float" << endl;

%%

```
%%
main()
{yylex();}
{yywrap();}
{ ... }
%
```

3. Kısım

ÖRNEK:

```
INT[0-9]+
EXP([eE][+-]?{INT})
IDENT[a-zA-Z][a-zA-Z0-9]*
%{int i;
  float f;%}
```

} 1. Kısım Lexical fonksiyonlar ve C bildirimleri

```
%%if    printf("keyword-if\n");
while printf("keyword-while\n");
else  printf("keyword-else\n");
{IDENT} printf("Identifier\n");
\+|\-|\*|\/ printf("arithmetic op\n");
"="   printf("Assign op\n");
==|<|>|<=|>=|!= printf("relop\n");
{INT}\.{INT}?{EXP}? {sscanf(yytext,"%f",&i);
                     printf("float\n");}
%%
```

2. Kısım

```
%%
main()
{yylex();}
yywrap()
{ ... }
%
```

3. Kısım

LEX programı LEX.yy.c adlı dosya oluşturur, bu dosya C-derleyicisi ile derlenir, YACC programı tarafından çağrılır.

07.03.2019 (5)

!! vizede automata <=> C-kod 20 puan

regüler ifade → automata

ödev: 50 puan Flex, 50 puan Bizon. Finalden 2 hafta önce

! vizede LEX ortaminda yazmak bir problemi

14.03.2019 (6)

Syntax Analysis (Sözdizim Analizi)

```
x = (2+3) * 9);   // mismatched parenthesis
if (x>y) x==2;    // missing parenthesis
while (x==3) do f1(x);   // invalid keyword
```

Lexical analiz: sözcükler dile ait midir?
Syntax analiz: sözcükler dilin gramer kurallarına uygun şekilde bir araya gelmiş mi?

Gramerler

1)
1. $S \Rightarrow 050$
2. $S \Rightarrow 151$
3. $S \Rightarrow 0$
4. $S \Rightarrow 1$

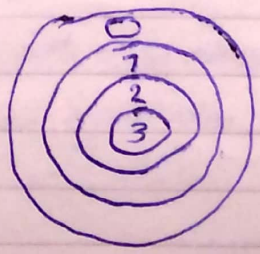
$S \Rightarrow 050 \Rightarrow 01510$
$\Rightarrow 0105010$
$\Rightarrow 0101010$

2)
$S \Rightarrow ASB$
$S \Rightarrow \varepsilon$
$A \Rightarrow a$
$B \Rightarrow b$

$S \Rightarrow ASB \Rightarrow aSB$
$\Rightarrow aASBB$
$\Rightarrow aaASBBB$
$\Rightarrow aaaSBBB$
$\Rightarrow aaa\varepsilon BBB$
$\Rightarrow aaa\varepsilon bBB$
$\Rightarrow aaa\varepsilon bbB$
$\Rightarrow aaa\varepsilon bbb$

Gramer türleri

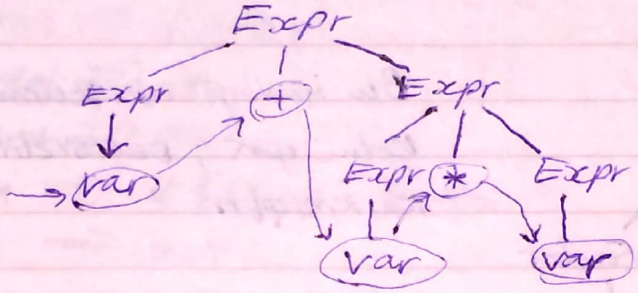
0 - $\alpha S \beta \Rightarrow \alpha \gamma \beta$ {Sınırsız diller}
1 - $\alpha A \beta \Rightarrow \alpha \gamma \beta$ {CSL}
2 - $A \Rightarrow \alpha$ {CFL}
3 - $A \Rightarrow aB$ {Regüler dil / Right Linear Language}



Belirsizlik

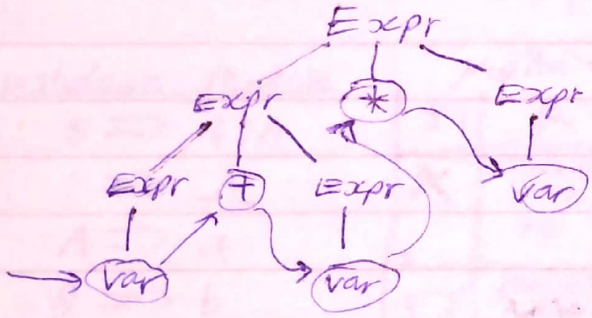
G: Expr → Expr + Expr
Expr → Expr * Expr
Expr → (Expr)
Expr → var
Expr → const

var + var * var



30

var = 5 ise

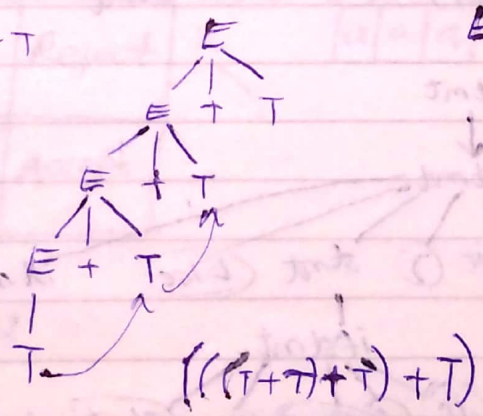


50

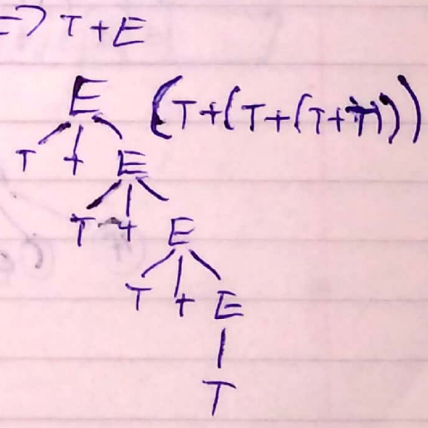
Belirsizlik var

Belirsizliği ortadan kaldırmak için belirsizliğin oluştuğu noktaya yeni bir non-terminal tanımlarız.

E => E + T
E => T



E => T + E

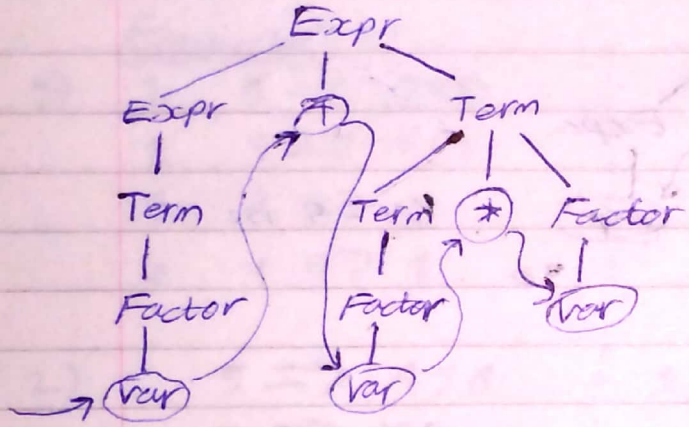


E => E + E

Tehlike

G2: Expr => Expr + Term
Expr => Term
Term => Term * Factor
Term => Factor
Factor => (Expr)
Factor => var
Factor => const

var + var * var

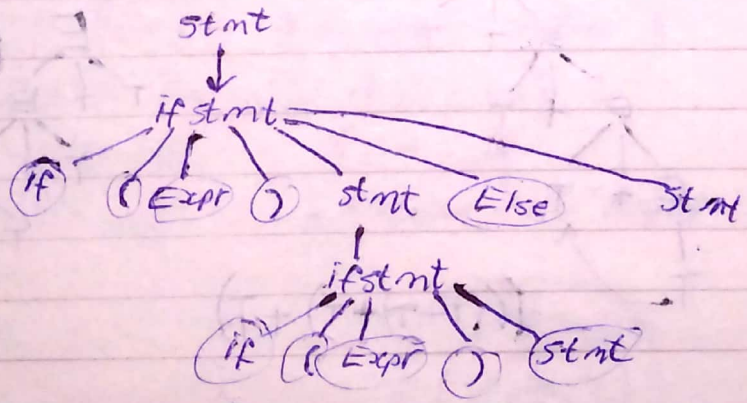


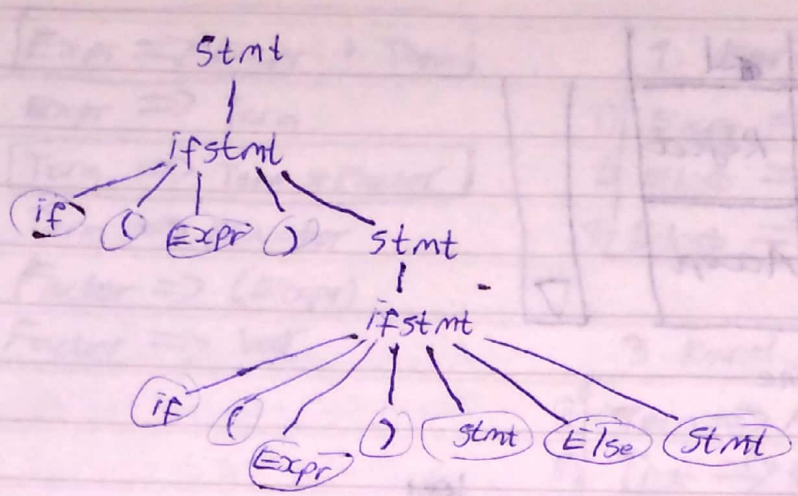
Bu ifadeyi oluşturmanın başka yolu yok, belirsizlik ortadan kalkmıştır.

ÖRNEK:

G: 1) stmt => Ifstmt
2) Ifstmt => if (Expr) stmt
3) ifstmt => if (Expr) stmt else stmt

if (Expr) if (Expr) stmt else stmt





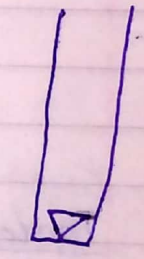
G2: 1) Stmt ⇒ ifstmt
2) ifstmt ⇒ M
3) ifstmt ⇒ M2
4) M ⇒ if (Expr) M else M
5) M ⇒ stmt 2
6) M2 ⇒ if (Expr) stmt
7) M2 ⇒ if (Expr) M else M2

Pushdown Makine

G: S ⇒ ASB
S ⇒ Λ
A ⇒ a
B ⇒ b

Yığıta eleman ekleme durumu

S1	a	b	↵
X	Push(a) ilerle S1	pop ilerle S2	Reject
▽	Push(a) ilerle S1	Reject	Accept

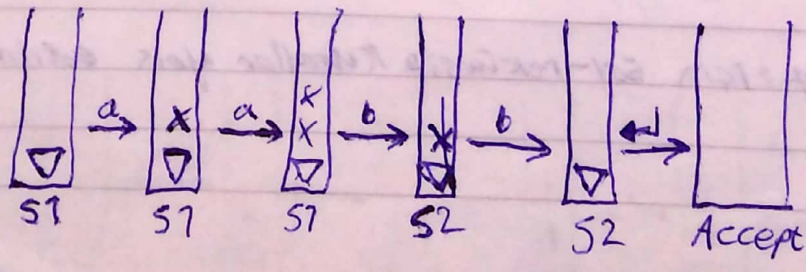


Yığıttan eleman çıkarma durumu

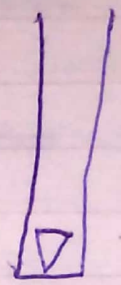
S2	a	b	↵
X	Reject S1	pop ilerle S2	Reject
▽	Reject	Reject	Accept

a	a	a	a	b	b	b	b	

W = a a b b ↵



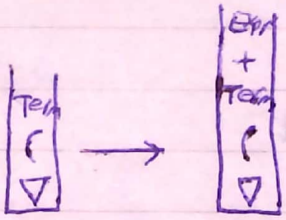
S1	(	)	↵
X	Push(X) ilerle S1	POP ilerle S1	Reject
▽	Push(X) ilerle S1	Reject	Accept



push down makine
(,) dengesi

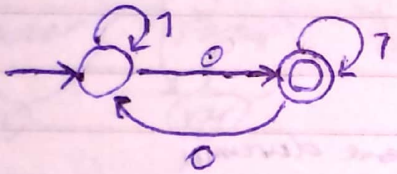
Exp ⇒ Exp + Term

Rep(Term, +, Expr)



ÖRNEK

Tek sayıda 0 içeren karakter katarları



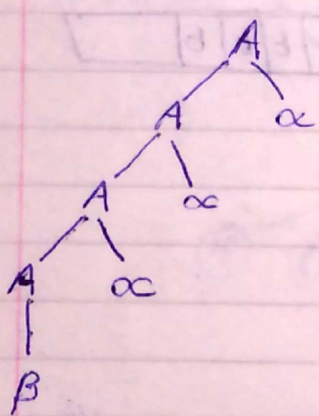
S ⇒ 0
S ⇒ 1S
S ⇒ 0A
A ⇒ 1
A ⇒ 1A
A ⇒ 0S

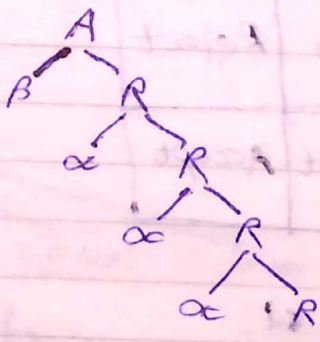
Topdown parsing

A ⇒ Aα
A ⇒ β

yerine →

A ⇒ βR
R ⇒ αR
R ⇒ ε





Topdown parsing için sol-rekürsif kurallar yok edilmelidir.

1) Sol rekürsif kural

1. Expr ⇒ Expr + Term
2. Expr ⇒ Term
3. Term ⇒ Term * Factor
4. Term ⇒ Factor

Factor ⇒ (Expr)

Factor ⇒ var

1. kural yerine

1) Expr ⇒ Term Elist
2) Elist ⇒ + Term Elist
3) Elist ⇒ ε

3. kural yerine

4) Term ⇒ Factor Tlist
5) Tlist ⇒ * Factor Tlist
6) Tlist ⇒ ε

sol rekürsiften kurtardık

7) Factor ⇒ (Expr)
8) Factor ⇒ var

2) Factorization

$A \Rightarrow \alpha\beta \mid \alpha\gamma$

Şeritten α okununca hangi kuralın çalıştırılacağı belli değil.

yerine

$A \Rightarrow \alpha X$

$X \Rightarrow \beta$

$X \Rightarrow \gamma$

S ⇒ E+S

S ⇒ E

E ⇒ sayı

E ⇒ (S)

S ⇒ ES'

S' ⇒ +S

S' ⇒ ε

E ⇒ sayı | (S)

21.03.2019 (7)

Muhammed Hash

Topdown Parsing

- Sol rekürsif Kurallar
- Kararsız kurallar
- Basit Gramer

$A \Rightarrow a\alpha \quad a \in V_T \quad \alpha \in (V_N \cup V_T)^+$

→ Topdown parsing'de basit gramer kesinlikle Terminal simge ile başlar

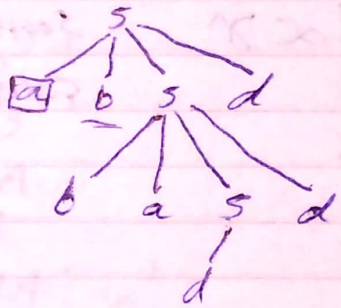
$S \Rightarrow aSb$	$S \Rightarrow aSb$	$S \Rightarrow aSb$
$S \Rightarrow b$	$S \Rightarrow \varepsilon$	$S \Rightarrow a$
Basit gramer	basit gramer değil	Basit gramer değil

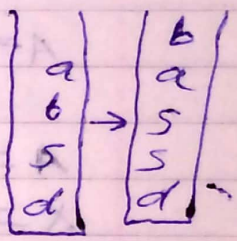
ilkler kümelerinin kesişimi boş küme değil

G:

1. $S \Rightarrow abSd$
2. $S \Rightarrow baSd$
3. $S \Rightarrow d$

$x = abbaddd$





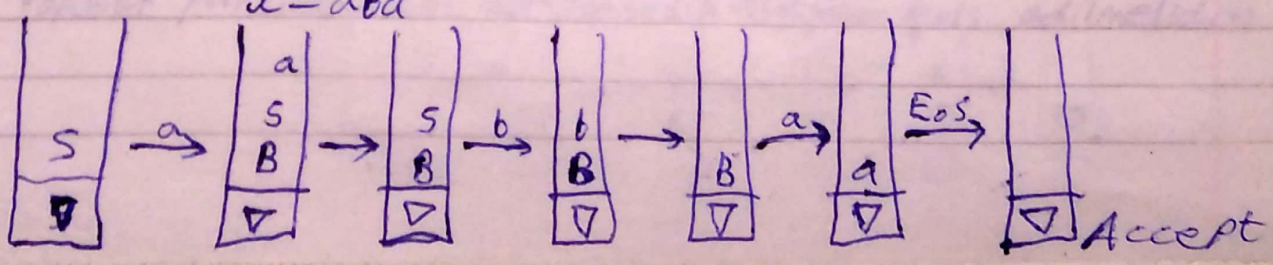
Pushdown Makine

ÖRNEK:

1. $S \Rightarrow aSB$
2. $S \Rightarrow b$
3. $B \Rightarrow a$
4. $B \Rightarrow bBa$

	a	b	EoS
S	Rep(BSa) Retain	Rep(b) Retain	Reject
B	Rep(a) Retain	Rep(aBb) Retain	Reject
a	Pop Advance	Reject	Reject
b	Reject	Pop Advance	Reject
▽	Reject	Reject	Accept

$x = aba$



Recursive Descent Parser

```
char inp;
void parse()
{
  cin>> inp;
  S();
  if(inp == "EOS") Accept();
  else Reject();
}

Void S(){

  if(input == 'a')
   {
     cin >> inp;
     S();
     B();
   }
  else if (inp == b) cin>>inp;
  else Reject();
}

Void B()
{
  if(inp == 'a') cin>> inp;
  else if (inp == 'b')
  {
    cin >>inp;
    B();
    if(inp == 'a') cin >> inp;
    else Reject();
  }
```

Yani Basit Gramer

$A \Rightarrow \varepsilon$

G:

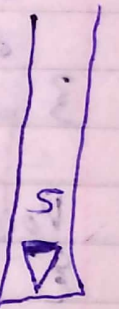
1. $S \Rightarrow aAS$
2. $S \Rightarrow b$
3. $A \Rightarrow cAS$
4. $A \Rightarrow \varepsilon$

$S \Rightarrow aAS \Rightarrow acASS \Rightarrow acSS \Rightarrow acbS \Rightarrow acbaAS \Rightarrow acbaS \Rightarrow acbab$

follow(S) = { Eos, a, b }

follow(A) = { a, b }

	a	b	c	Eos
S	Rep(SAd) Retain	Rep(b) Retain	Reject	Reject
A	Pop Retain	Pop Retain	Rep(SAc) Retain	Reject
a	Pop Advance	Reject	Reject	Reject
b	Reject	Pop Advance	Reject	Reject
c	Reject	Reject	Pop Advance	Reject
▽	Reject	Reject	Reject	Accept



Recursive Descent Parser

↓

Program

First (α) = ?

$\alpha = XYZ$ ise

First (α) = X eğer $X \in V_T$

aksi halde

First (α) = First (X) eğer $\varepsilon \notin$ first (X)

Eğer $\varepsilon \in$ First (X) ise First (α) = [First (X) − ε] ∪ First (YZ)

First (YZ) = Y eğer $Y \in V_T$

değilse First (YZ) = First (Y) eğer First (Y) $\in$ içermiyorsa

eğer $\varepsilon \in$ First (Y) ise First (YZ) = [First (Y) − ε] ∪ First (Z)

ÖRNEK:

$S \Rightarrow ACB \mid CbB \mid Ba$

$A \Rightarrow da \mid BC$

$B \Rightarrow g \mid \lambda$

$C \Rightarrow h \mid \lambda$

$V_T = \{a, b, d, g, h\}$

first (S) = first (ACB) ∪ first (CbB) ∪ first (Ba)

first (A) = first (da) ∪ first (BC)

first (B) = first (g) ∪ first (ε) = $\{g, \varepsilon\}$

first (C) = $\{h, \varepsilon\}$

first (BC) = first (B) ∪ first (C) = $\{g, h, \varepsilon\}$

first (A) = $\{d, g, h, \varepsilon\}$

first (ACB) = (first (A) − ε) ∪ first (CB)

= $\{d, g, h\}$ ∪ (first (C) − ε) ∪ first (B)

= $\{d, g, h, \varepsilon\}$

first (CbB) = (first (C) − ε) ∪ first (bB)

= $\{h\}$ ∪ $\{b\}$ = $\{h, b\}$

first (Ba) = (first (B) − ε) ∪ first (a) = $\{g, a\}$

first (S) = $\{d, g, h, \varepsilon, b, a\}$

Follow Kümesi

Kural 1: S, bir başlangıç NT ise
($E_0 S$), $, ↵
($E_0 S$)'i follow kümesine ekle

Kural 2: $A \Rightarrow \alpha B \beta$ iken
$B \Rightarrow \varepsilon$ kuralı varsa
First(β)'daki bütün singleları (ε hariç)
follow(B)'ye ekle

Kural 3: $A \Rightarrow \alpha B$
follow(B)'ye follow(A) eklenir.

ÖRNEK:

$E \Rightarrow TE'$
$E' \Rightarrow +TE' \mid \varepsilon$
$T \Rightarrow FT'$
$T' \Rightarrow *FT' \mid \varepsilon$
$F \Rightarrow (E) \mid id$

Bu gramerin Follow kümelerini bulun.

First(E) = First(T) = First(F) = {(, id}
First(E') = {+, ε}
First(T') = {*, ε}

follow(E) = {$,)}
follow(E') = follow(E)
follow(T) = follow(T') = {+, $,)}
follow(F) = {+, $,), *}

04.04.2019 (9)

Lookahead Symbol (ileri bakış simgesi)

LL (1)
↳ Leftmost derivation
↳ Left to Right

ÖRNEK:

$E \Rightarrow TX$
$T \Rightarrow (E) \mid int\ Y$
$X \Rightarrow +E \mid \varepsilon$
$Y \Rightarrow *T \mid \varepsilon$

1) FIRST(E) = { int, (}
FIRST(T) = { (, int }
FIRST(X) = { +, ε }
FIRST(Y) = { *, ε }

2) Follow(E) = { $,) }
Follow(X) = Follow(E) = { $,) }
Follow(T) = Follow(Y) = { +, follow(F) } = { +, $,) }
Follow(Y) = { +, $,) }

	int	*	+	(	)	$
E	→TX	error	error	→TX	error	error
T	→intY	error	error	→(E)	error	error
X	error	error	→+E	error	→ε	→ε
Y	error	→*T	→ε	error	→ε	→ε

Finalde 4 soru, herbiri 25 puan

ÖRNEK: (LL(1) gramer)

(Recursive Descent) tablosu:

1) Herbir terminal simge ve ($\dashv$) için sütün
   Herbir nonterminal simge ve (∇) için satır
2) Her $A \Rightarrow a\alpha$ kuralı için
   ($A \times a$) hücresine $Rep(\alpha^R a)$ yaz
3) $\begin{pmatrix} a \times a \\ b \times b \end{pmatrix}$ hücrelerine (POP/ilerle) yaz
4) ($\nabla \times \dashv$) hücresine ACCEPT yaz
5) Her $A \Rightarrow \varepsilon$ kuralı için A satır ve follow(A) kümesindeki sütunlara (pop/retain) yaz
6) Diğer bütün sütunlara REJECT yaz
7) Yığıtı $\begin{array}{|c|}\hline S \\ \hline \nabla \\ \hline \end{array}$ ile başlat

$S \Rightarrow ABc$ first(S) = {b, c} first(B) = {c}

$A \Rightarrow bA$ first(A) = {b, ε}

$A \Rightarrow \varepsilon$ follow(A) = {c}

$B \Rightarrow c$

	b	c	$\dashv$
S	Rep(CBA) Retain	Rep(CBA) Retain	X error
A	Rep(Ab)	Pop Retain	X error
B	X error	Rep(C)	X error
b	Pop ilerle	X error	X error
c	X error	Pop ilerle	X error
∇	X error	X error	ACCEPT

ÖRNEK:

1) Expr ⇒ Expr + Term
2) Expr ⇒ Term
3) Term ⇒ Term * Factor
4) Term ⇒ Factor
5) Factor ⇒ (Expr)
6) Factor ⇒ var

Bu gramer için top-down parser tasarlayınız.
Sol rekürsiyona yer olmamız gerekiyor:

$A \Rightarrow A\alpha$
$A \Rightarrow \beta$
} yerine $A \Rightarrow \beta R$, $R \Rightarrow \alpha R$, $R \Rightarrow \varepsilon$

1) kuralı yerine

1) Expr ⇒ Term Elist
2) Elist ⇒ + Term Elist
3) Elist ⇒ ε

3) kuralı yerine

4) Term ⇒ Factor Tlist
5) Tlist ⇒ * Factor Tlist
6) Tlist ⇒ ε

7) Factor ⇒ Expr
8) Factor ⇒ var

Kural 1) first(Expr) = First(Term Elist) = {(, var}
first(elist) = first(+ Term Elist) = {+} ∪ first{ε} = {+, ε}
follow(Elist) = {), ↵}
first(Term) = first(Factor Tlist) = {(, var}
first(Tlist) = First(* Factor Tlist) = {*}
follow(Tlist) = {+,), ↵}
first(factor) = {(, *}

	+	*	(	)	var	$
Expr	X	X	Rep(Elist Term) Retain	X	Rep(Elist Term) Retain	X
Elist	Rep(Elist Term+) Retain	X		Pop Retain		Pop Retain
Term	X	X	Rep(Tlist Factor) Retain	X	Rep(Tlist Factor) Retain	X
Tlist	Pop/Retain	Rep(Tlist Factor*) Retain	X	Pop/Retain	X	Pop/Retain
Factor	X	X	Rep( )EXPT( )	X	Rep(var) Retain	X
+	Pop/ilerle	X	X	X	X	X
*	X	Pop/ilerle	X	X	X	X
(	X	X	Pop/ilerle	X	X	X
)	X	X	X	Pop/ilerle	X	X
var		X	X		Pop/ilerle	X
▽	X	X	X	X	X	ACCEPT

11.04.2019 (10)

Syntax Directed Translation (Sözdizim Yönlü Çeviri):

$A \Rightarrow \{action\}$ → çeviri grameri

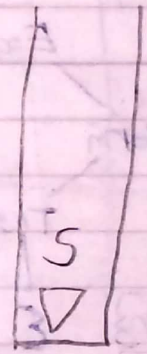
ÖRNEK:

G: $S \Rightarrow \{print\}\ aS$

$S \Rightarrow bB$

$B \Rightarrow \{print\}$

	a	b	↵
S	Rep(Sa{print}) Retain	Rep(Bb)	
B			Rep({print}) Retain
a	Pop Advance		
b		Pop Advance	
{print}	Pop Retain out({print})	Pop Retain out({print})	Pop Retain out({print})
▽			Accept




```
Void S()
{ if (input == a)
  { cin >> inp;          kural 1
    cout << 'print';
    S();                 end kural 1 }
  }
  else if (inp == b)
  { cin >> inp;          kural 2
    B();
  }
  else Reject();         end kural 3
}
```

```
Void B()
{ if (inp == '↵')        kural 3
    cout << "print";
  else Reject();         end kural 3
```

ÖRNEK:

1. Expr ⇒ Terim Elist

Elist ⇒ +Term {+} Elist

Elist ⇒ ε

Term ⇒ Factor Tlist

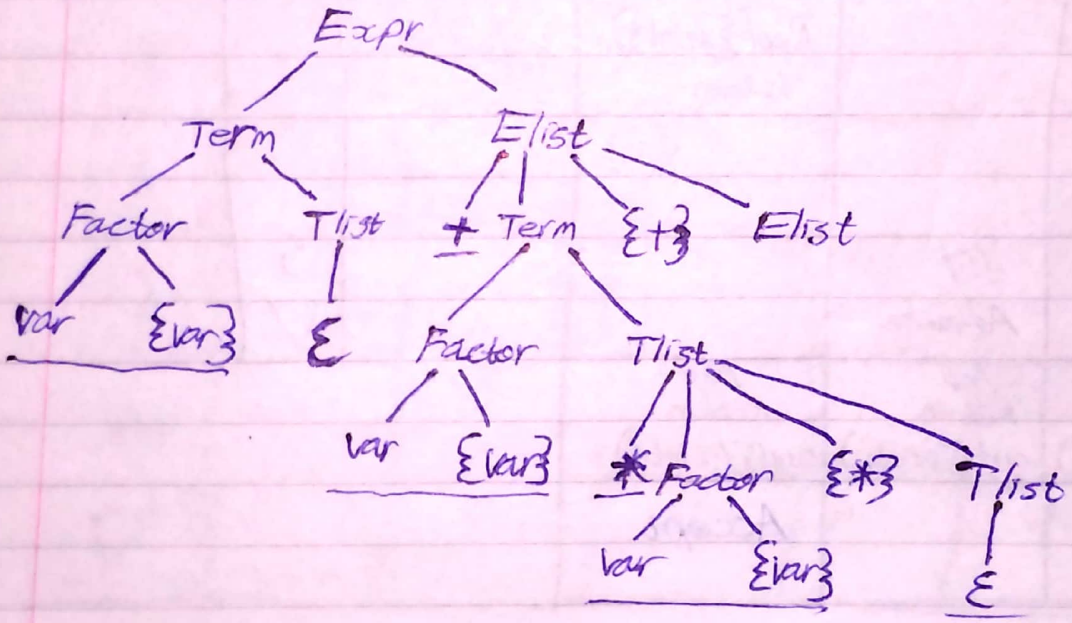
Tlist ⇒ *Factor {*} Tlist

Tlist ⇒ ε

Factor ⇒ Expr

Factor ⇒ var {var}

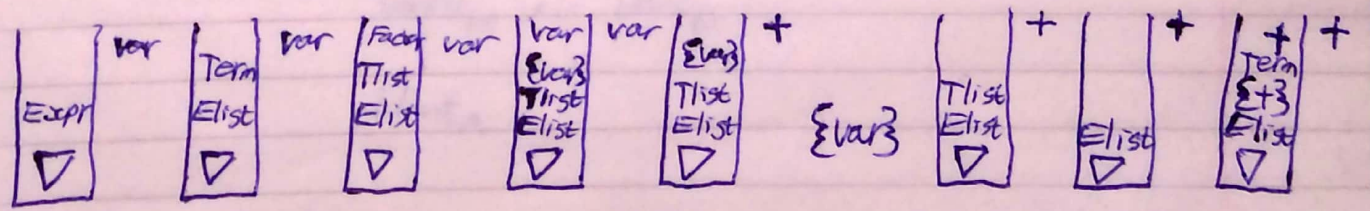
var + var * var

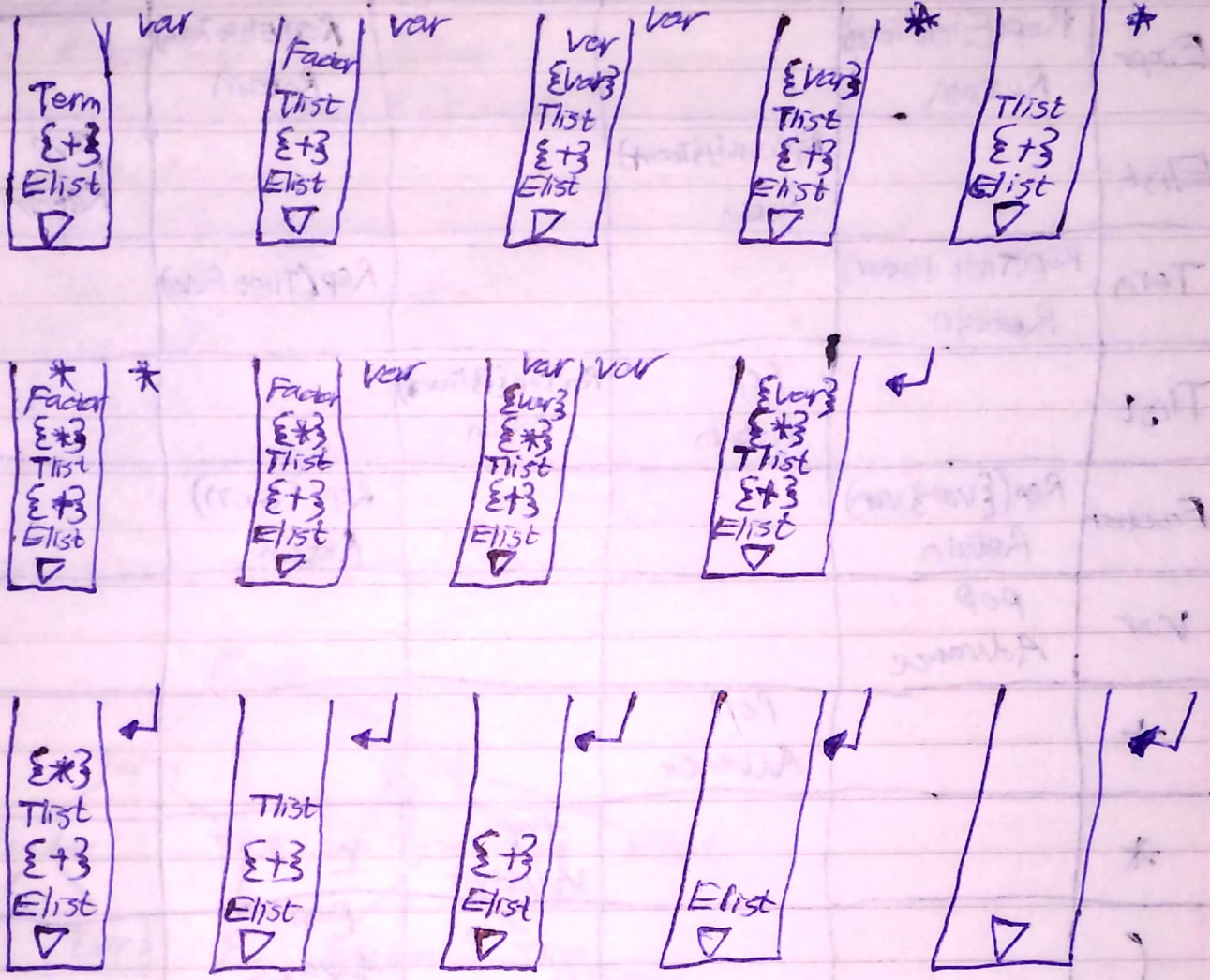


var {var} + var {var} * var {var} {*} {+}
{var} {var} {var} {*} {+}

	var	+	*	(	)	↵
Expr	Rep(Elist Term) Retain			Rep(Elist Term) Retain		
Elist		Rep(Elist {+} Term+) Retain			pop Retain	pop Retain
Term	Rep(Tlist Factor) Retain			Rep(Tlist Factor)		
Tlist		pop Retain	Rep(Tlist {*} Factor*) Retain			
Factor	Rep({var} var) Retain			Rep((Expr)) Retain		
var	pop Advance					
+		pop Advance				
*			pop Advance			
(				pop Advance		
)					pop Advance	
{var}	pop Retain out (var)	out {var}				pop Retain out {var}
{+}	pop Retain out (+)	out {+}				pop Retain out {+}
{*}	pop Retain out (*)	out {*}				pop Retain out {*}
▽						Accept

var +var *var




var
Term
{+}
Elist
var
Factor
Tlist
{+}
Elist
var
Ver
{var}
Tlist
{+}
Elist
*
{var}
Tlist
{+}
Elist
*
Tlist
{+}
Elist
*
Factor
{*}
Tlist
{+}
Elist
var
Factor
{*}
Tlist
{+}
Elist
var
var
{var}
{*}
Tlist
{+}
Elist
{var}
{*}
Tlist
{+}
Elist
{*}
Tlist
{+}
Elist
Tlist
{+}
Elist
{+}
Elist
Elist

~~Attr~~

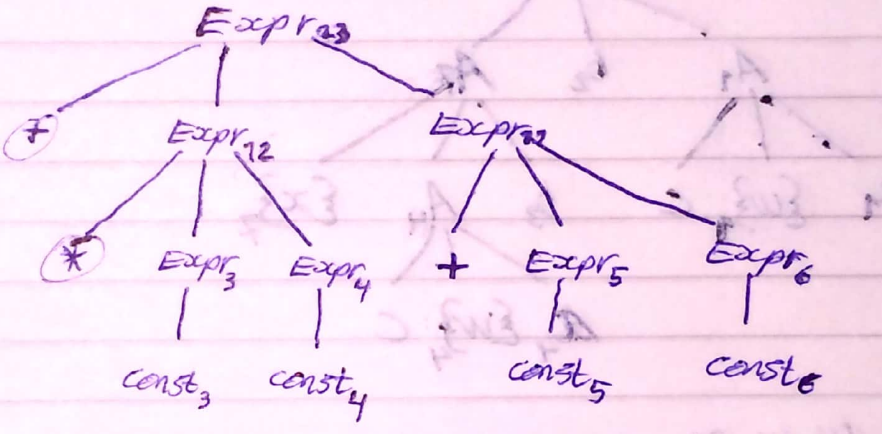
Attributed Grammars (Nitelikli gramerler)

1) Sentezlenmiş Nitelik
2) Edinilmiş Nitelik

ÖRNEK:

1) $Expr_p \Rightarrow +Expr_q\ Expr_r$ $\quad p \leftarrow q+r$
2) $Expr_p \Rightarrow *Expr_q\ Expr_r$ $\quad p \leftarrow q*r$
   $Expr_p \Rightarrow const_q$ $\quad p \leftarrow const_q$

input: + * 34 + 56



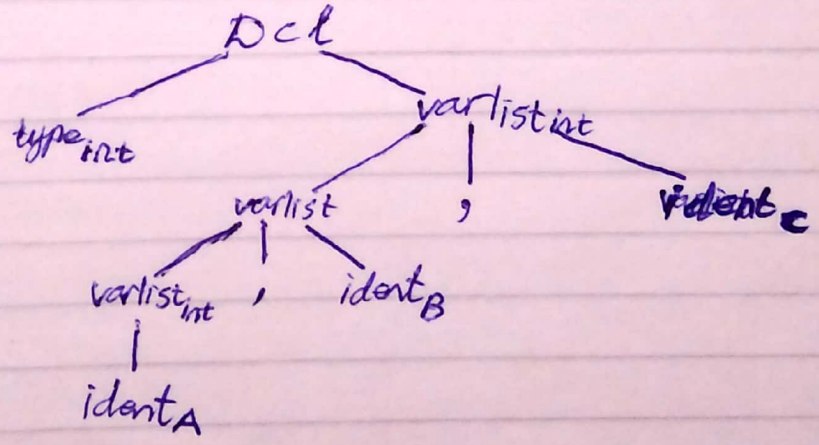
NOT: Sentezlenmiş nitelik değerini çocuk düğümlerden alır.

ÖRNEK:

Gramer

1) $Dcl \Rightarrow type_t\ Varlist_v$ $\quad V \leftarrow t$
2) $Varlist_v \Rightarrow Varlist_w\ ,\ ident_x$ $\quad W \leftarrow V$
3) $Varlist_v \Rightarrow ident_x$

input: int A, B, C



ÖRNEK:

$S_p \Rightarrow A_q\ B\ r\ A_t \qquad p \Leftarrow r+t$

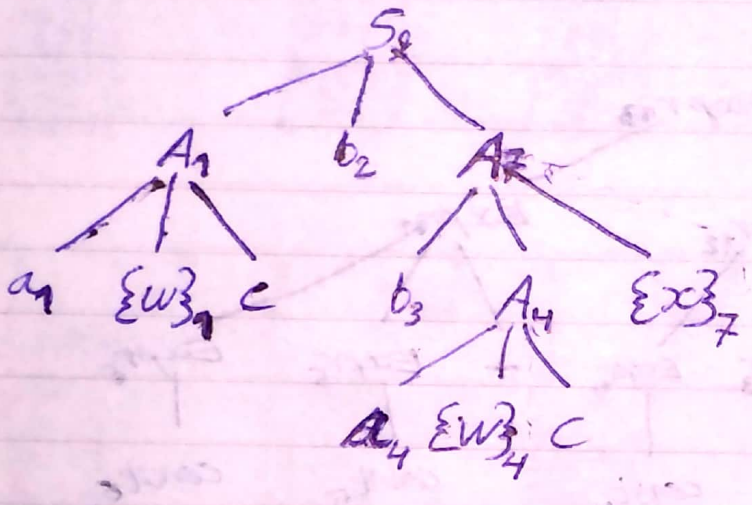
$A_p \Rightarrow a_p \{w_p\} C$

$A_p \Rightarrow b_q\ A_r\ \{x\}_p \qquad p \Leftarrow q+r$

!! input = $a_1 C\ b_2\ b_3\ a_4\ C$ ise

a) Nitelikli gramer ağacını oluşturunuz

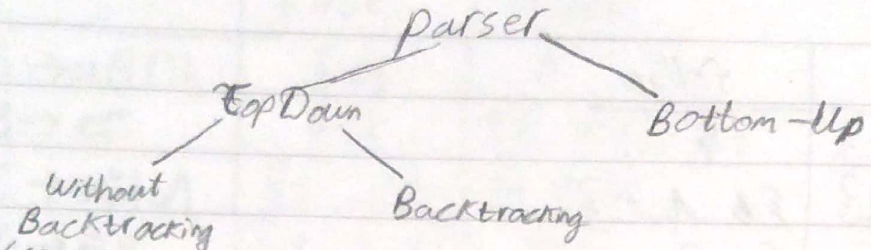
b) Bu gramere ilişkin çıkış sembolü nedir?

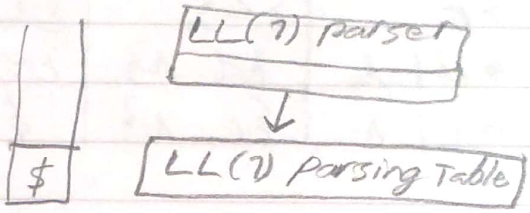


$w_1\ w_4\ x_7$

ÖRNEK:

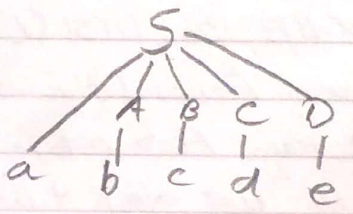
25.04.2019 (11)





ÖRNEK:

$S \rightarrow aABCD$
$A \rightarrow b$
$B \rightarrow c$
$C \rightarrow d$
$D \rightarrow e$



$S \rightarrow aABCD \rightarrow abBCD \rightarrow abcCD$
$\rightarrow abcdD \rightarrow abcde$

	first	follow
$S \rightarrow aABCD$	$\{a\}$	$\$$
$A \rightarrow b$	$\{b\}$	$\{c\}$
$B \rightarrow c$	$\{c\}$	$\{d\}$
$C \rightarrow d$	$\{d\}$	$\{e\}$
$D \rightarrow e$	$\{e\}$	$\$$

ÖRNEK:

	first	follow
$S \rightarrow ABCD$	$\{b, \Lambda, c\}$	$\$$
$A \rightarrow b \mid \Lambda$	$\{b, \Lambda\}$	$\{c\}$
$B \rightarrow c$	$\{c\}$	$\{d\}$
$C \rightarrow d$	$\{d\}$	$\{e\}$
$D \rightarrow e$	$\{e\}$	$\$$

ÖRNEK

	first	follow
$S \to ABCDE$	$\{a, b, c\}$	\$
$A \to a \mid \Lambda$	$\{a, \Lambda\}$	$\{b, \Lambda, c\}$
$B \to b \mid \Lambda$	$\{b, \Lambda\}$	$\{c\}$
$C \to c$	$\{c\}$	$\{d, \$, e\}$
$D \to d \mid \Lambda$	$\{d, \Lambda\}$	$\{e, \$\}$
$E \to e \mid \Lambda$	$\{e, \Lambda\}$	\$

Follow(α):

- S başlangıç ise \$ simgesini follow'a ekle
- $A \to \alpha B \beta$ ise first(β) kümesinin Λ dışındaki tüm elemanlarını Follow(B)'ye ekle * follow(B) = first(β)
  $A \to \alpha B$ veya $A \to \alpha B \beta$ ve $\Lambda \in$ first(B) ise follow(A) kümesinde yer alan tüm elemanları follow(B)'ye ekle
  $A \to \alpha B$ veya $A \to \alpha B \beta$ $\beta^* \to \Lambda$
  * follow(B) = follow(A) ekle

ÖRNEK:

$S \to ACB \mid CbB \mid Ba$
$A \to da \mid BC$
$B \to g \mid \Lambda$
$C \to h \mid \Lambda$

	first	follow
S	$\{d, g, h, \Lambda, b, a\}$	\$
A	$\{d, g, h, \Lambda\}$	$\{h, g, \$\}$
B	$\{g, \Lambda\}$	$\{\$, a, h, g\}$
C	$\{h, \Lambda\}$	$\{g, \$, b, h\}$

ÖRNEK:

	first	follow
$S \to aABb$	$\{a\}$	\$
$A \to c \mid \Lambda$	$\{c, \Lambda\}$	$\{d, b\}$
$B \to d \mid \Lambda$	$\{d, \Lambda\}$	$\{b\}$

ÖRNEK

	first	follow
$S \to aBDh$	$\{a\}$	$\$$
$B \to cC$	$\{c\}$	$\{h, g, f\}$
$C \to bC \mid \Lambda$	$\{b, \Lambda\}$	$\{h, g, f\}$
$D \to EF$	$\{g, \Lambda, f\}$	$\{h\}$
$E \to g \mid \Lambda$	$\{g, \Lambda\}$	$\{f, h\}$
$F \to f \mid \Lambda$	$\{f$	$\{h\}$

ÖRNEK:

	first	follow
$E \to TE'$	$\{id, ($	$\{\$,)\}$
$E' \to +TE' \mid \Lambda$	$\{+, \Lambda\}$	$\{\$,)\}$
$T \to FT'$	$\{id, ($	$\{+,), \$\}$
$T' \to *FT' \mid \Lambda$	$\{*, \Lambda\}$	$\{+,), \$\}$
$F \to id \mid (E)$	$\{id, ($	$\{+,), \$, *\}$

	id	+	*	(	)	$
E	$E \to TE'$			$E \to TE'$		
E'		$E' \to TE'$			$E' \to \Lambda$	$E' \to \Lambda$
T	$T \to FT'$			$T \to FT'$		
T'		$T' \to \Lambda$	$T' \to *FT'$		$T' \to \Lambda$	$T' \to \Lambda$
F	$F \to id$			$F \to (E)$		

02.05.2019 (T)

Lexical Analiz - Belirteçler doğruyor? Test edildi
Syntax // - // doğru sıradamı? Test edildi
↳ Graphviz oluşturuldu
Belirsizlik / Nondeterminizm kaldırıldı
Sol rekürsiyondan kurtarıldı

(Tahmine dayalı Ayrıştırma) LL(1) ile first-follow kümeleri belirlendi
Tahmine dayalı ayrıştırma tablosu oluşturuldu
Rekürsif inişli ayrıştırıcılı hazırlanır
↳ Graphviz kodu gerçekleştirildi

Konu) LR(0), LR(1) için Ayrıştırıcı (Parser) NFA'nın Oluşturulması

1-) S' → S Yeni bir NT(S') ve yeni bir kural ekle ve bunu root durumu yap
2-) A → --- • X --- X ∈ (VN ∪ VT)
X geçişiyle A → --- X • --- hangi duruma gideriz?
3-) A → --- X • Herhangi bir düşümde "•" biçiminde kural var ve bu kural o düşümdeki tek kural ise o düşüm bir kabul durumudur
4-) Yeni kural eklenemeyinceye kadar ve bütün kurallar sınanıncaya kadar bu işlemler devam edilir

Pushdown Automata Marked Rules

S → aAb biçiminde bir kural var ise,
soldan tanımaya başlarsak
S → •aAb (İşaretli kuralın ilk biçimi)
S → a•Ab } Ara biçimler
S → aA•b }
S → aAb• (İşaretli kuralın son biçimi)

işaret ... kümesinin kapanışı

* kural kümesi üzerinde çalışır.
* k^+ kümesine; * işaretli kurallar kümesinin kapanışı

Eğer işaretten sonra NT var ise bu NT'nin ilk biçimi kümeye ekleniş yeni kural işlem yapılmayacağa kadar kural eklemeye devam eder.

Soru

$S \to aS \mid A$
$A \to Ab \mid B$
$B \to Bc \mid C$
$C \to Cc \mid c$

k: { $S \to a.S$, $C \to .c$ }

k^+ (k kümesinin kapanışını bulunuz)

k^+ $S \to a.S$, $C \to .c$

$S \to .aS$, $S \to .A$

$A \to .Ab$, $A \to .B$

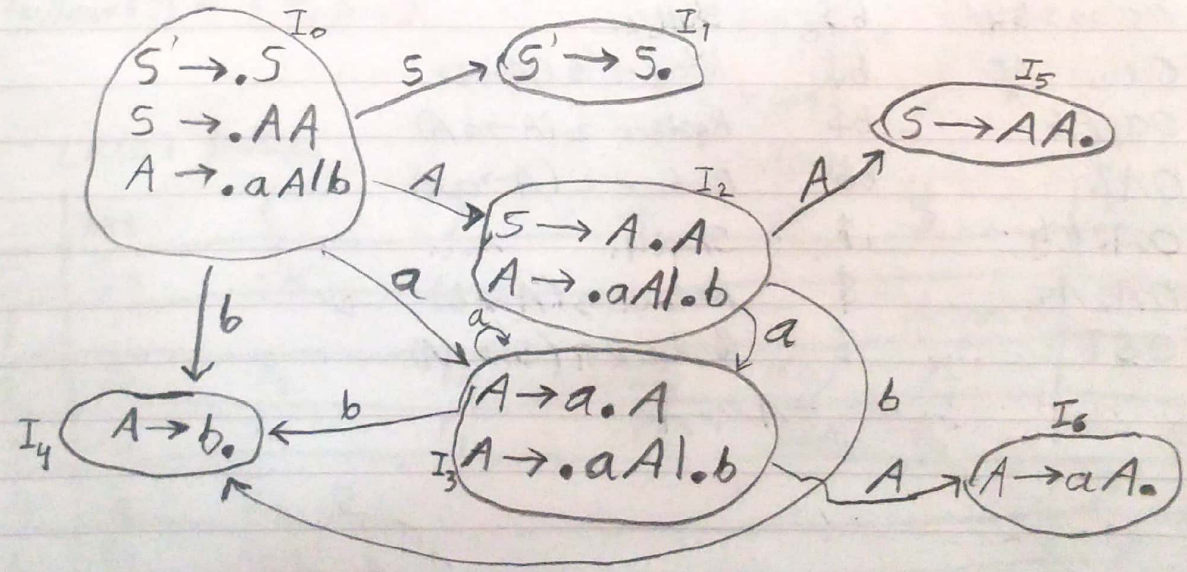
$B \to .Bc$, $B \to .C$

$C \to .Cc$

LR(0) Parsing Table

$S \to AA$ ①
$A \to aA$ ② $\mid b$ ③

$S' \to S$
$S \to AA$
$A \to aA \mid b$



	action			goto	
	a	b	$	A	S
0	S_3	S_4		2	1
1			Accept		
2	S_3	S_4		5	
3	S_3	S_4		6	
4	$r_3 r_3$	$r_3 r_3$	$r_3 r_3$		
5	r_1	r_1	$r_1 r_1$		
6	$r_2 r_2$	$r_2 r_2$	$r_2 r_2$		
7					

I_4 follow(A) = follow(S) ∪ follow(A)
$ a,b

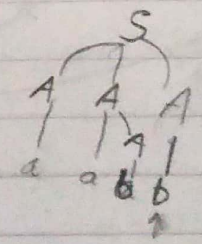
I_6 " " "

I_5 follow(S) = $

aabb?

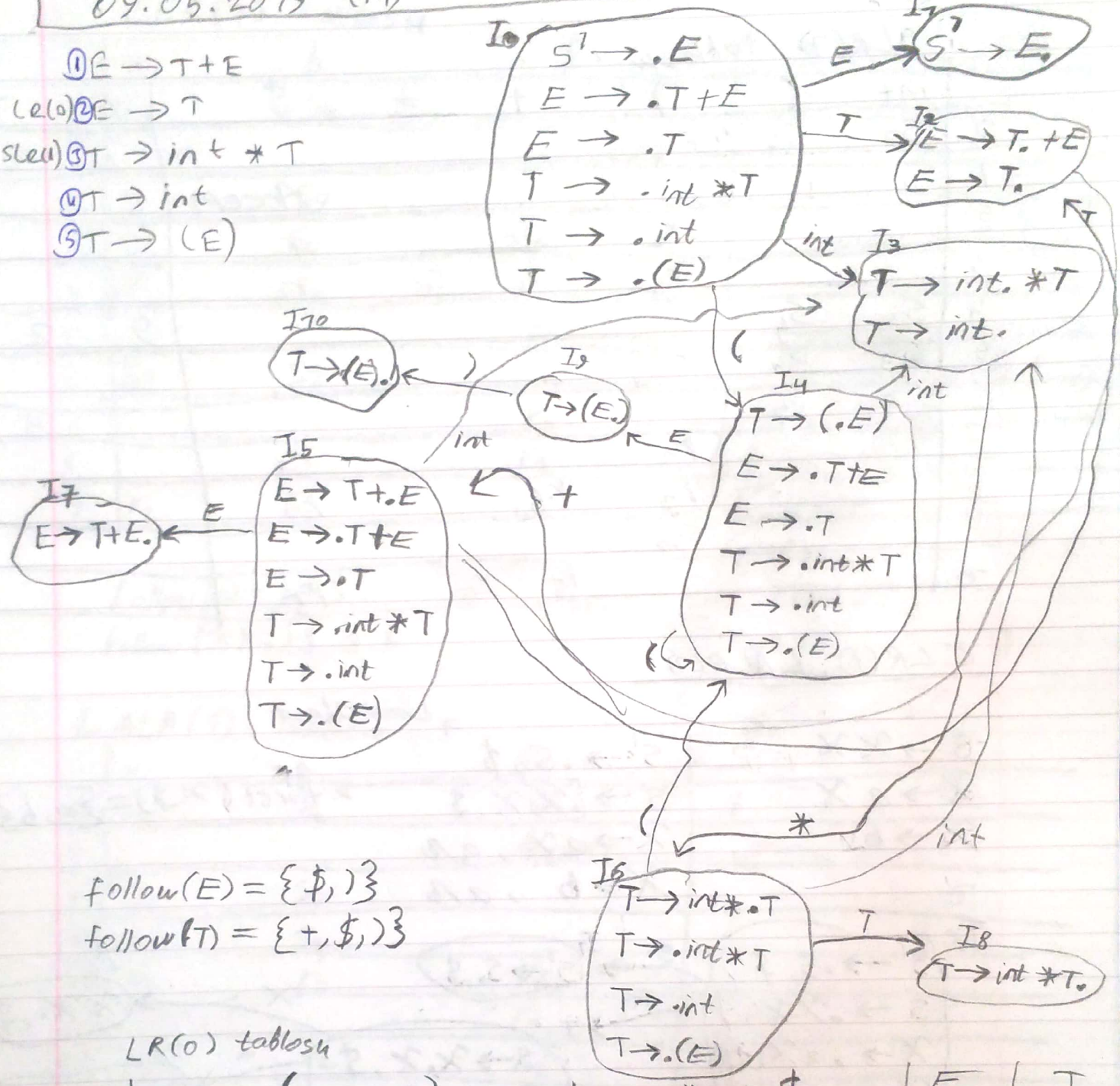
Stack	input	Action
0	aabb$	—
0a3	abb$	Shift 3
0a3a3	bb$	Shift 3
0a3a3b4	b$	Shift 4
0a3a3A6	b$	Reduce 3 (A→b)
0a3A6	b$	Reduce 2 (A→aA)
0A2	b$	Reduce 2 (A→aA)
0A2b4	$	Shift 4
0A2A5	$	Reduce 3 (A→b)
0S1	$	Reduce 1 (S→AA)

→ Accept



09.05.2019 (14)

① E → T+E
Le(o) ② E → T
sleu) ③ T → int * T
④ T → int
⑤ T → (E)



follow(E) = {$,)}
follow(T) = {+, $,)}

LR(0) tablosu

	int	(	)	+	*	$	E	T
0	S_3	S_4					1	2
1						ACCEPT		
2	r_2	r_2	r_2	r_2/S_5	r_2	r_2		
3	r_4	r_4	r_4	r_4	S_6/r_4	r_4		
4	S_3	S_4					9	2
5	S_3	S_4					7	2
6	S_3	S_4						8
7	r_1	r_1	r_1	r_1	r_1	r_1		
8	r_3	r_3	r_3	r_3	r_3	r_3		
9			S_{10}					
10	r_5	r_5	r_5	r_5	r_5	r_5		
	LR(0)		SLR(1)					

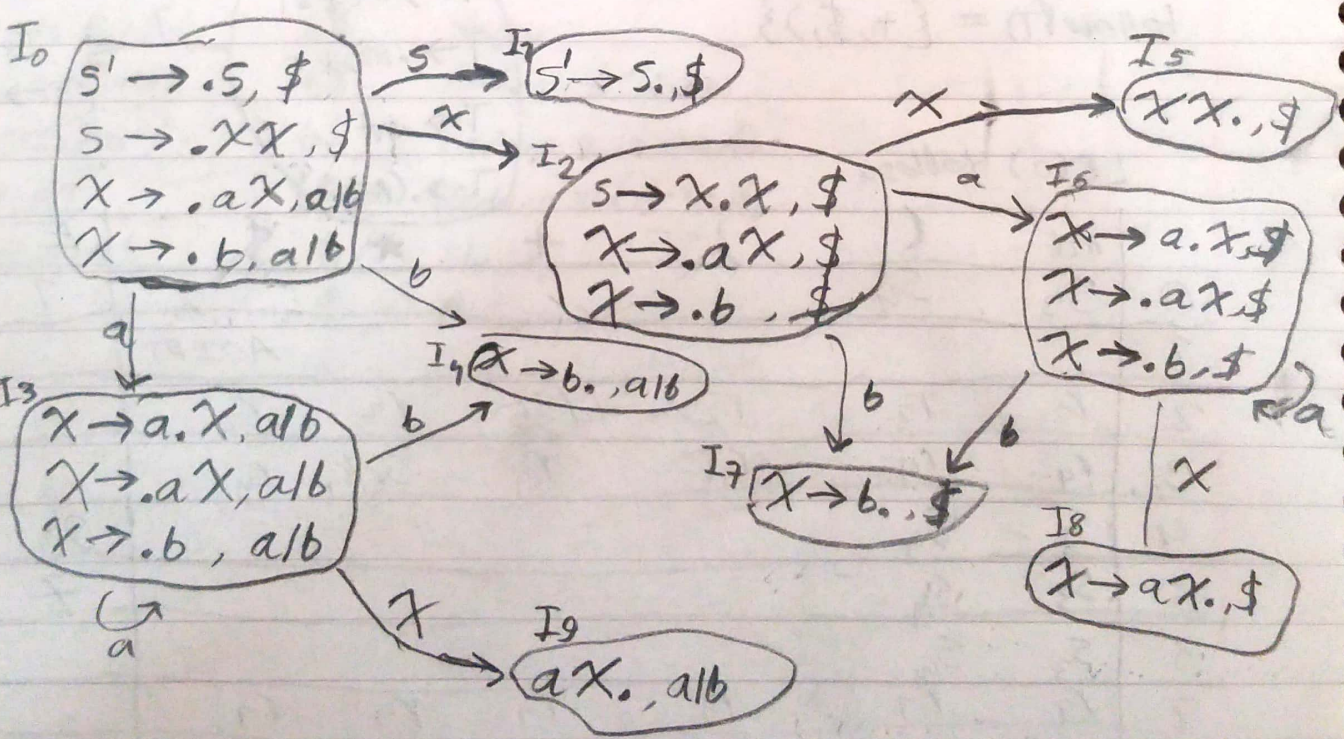
SLR(1) tablosu

	int	(	)	+	*	$	E	T
0	S_3	S_4					1	2
1						ACCEPT		
2			r_2	S_5		r_2		
3			r_4	r_4	S_6	r_4		
4	S_3	S_4					9	2
5	S_3	S_4						
6	S_3	S_4						8
7			r_1			r_1		
8			r_3	r_3		r_3		
9			S_{10}					
10				r_5	r_5	r_5		

CLR(1), LALR(1)

$S \to XX$
$X \to aX$
$X \to b$

$S' \to .S, \$$ ← Look ahead
$S \to .XX, \$$ → first($X\$$) = {a b}
$X \to aX, a/b$
$X \to b, a/b$



CLR(1) Tablsu

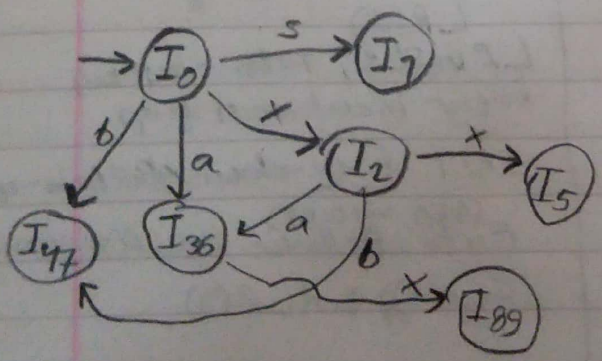
	a	b	$	S	X
0	S_3	S_4		1	2
1			Accept		
2	S_6	S_7			5
3	S_3	S_4			9
4	r_3	r_3			
5			r_1		
6	S_6	S_7			8
7			r_3		
8			r_2		
9	r_2	r_2			

follow(S) = {$}
follow(X) = {$, a, b

LALR(1) Tablosw

	a	b	$	S	X
0	S_3	S_4		1	2
1			Accept		
2	S_6	S_7			5
36	S_{36}	S_{47}			89
47	r_3	r_3	r_3		
5			r_1		
89	r_2	r_2	r_2		

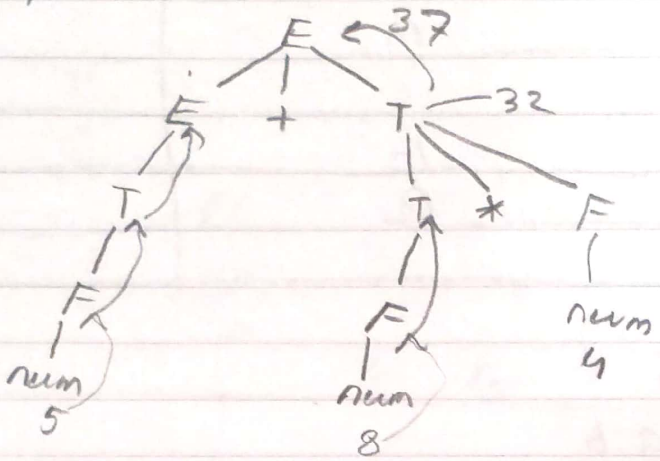
CLR(1) ≥ LR(0) = LALR(1) = SLR(1)



SDT (Syntax Directed Translation)

$E \to E + T \mid T$ { E.value = E.value + T value } { E.value = T.value }
$T \to T * F \mid F$ { T value = T value * F value } { T ... }
$F \to num$

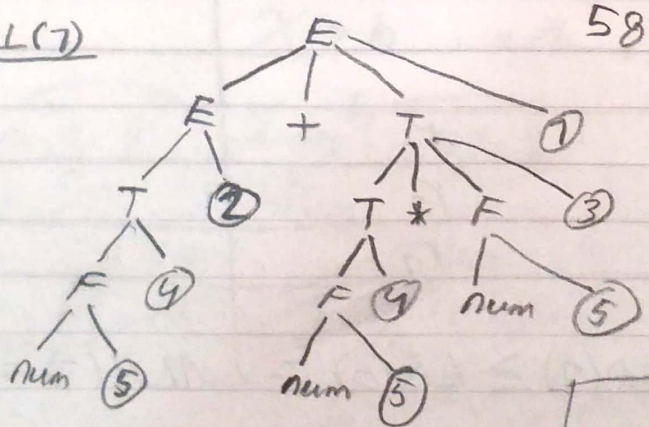
5+8*4



$E \to E + T \mid T$ ① { print("+") } ②
$T \to T * F \mid F$ ③ { print("*") } ④
$F \to num$ ⑤ { print(numval) }

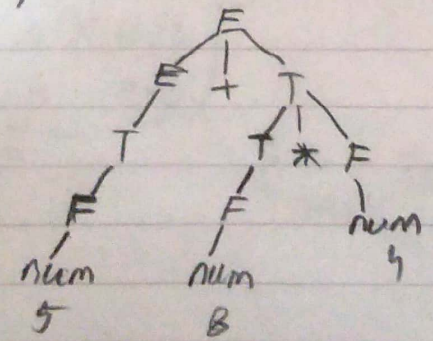
5+8*4

Topdown LL(1)



5 8 4 * T

Bottom-up LR



5 8 4 * + $

Finalde !!

LR(0)
LR ve SLR, Tablo oluştur, kalar incelemesi yap
SDT (Top-down / Bottom-up)
(NFA verecek)
First, follow (ve LR(1) parsing)
CLR(1)) LALR(1)